

# The University of Faisalabad

## **Lab Manual**

**AI-212  
Programming for AI**

**Submitted by**

**Muhammad Tayyab Imran      2023-BS-AI-019(5A)**

**Submitted to**

**Mr. Samraiz Zahid**

**Department of Computer Science**

---

## Table of Contents

---

|                                              |     |
|----------------------------------------------|-----|
| <b>Lab # 1:</b> Programming Fundamentals     | 1   |
| <b>Lab # 2:</b> OOP                          | 40  |
| <b>Lab # 3:</b> NumPy                        | 98  |
| <b>Lab # 4:</b> Assignment                   | 108 |
| <b>Lab # 5:</b> Seaborn, Matplotlib, Sklearn | 122 |

---

## Lab 1

### “Programming Fundamentals”

#### Program 1: Banking Transaction Validator with PIN

```
def withdrawal(balance):
    while True:
        amount = float(input("Enter withdrawal amount: "))
        if amount <= 0:
            print("Amount must be positive!")
            continue
        if amount > balance:
            print(f"Insufficient balance!")
            continue

        return amount

def service_charge(amount):
    return amount * 0.02

def print_receipt(initial_balance, amount, service_charge):
    final_balance = initial_balance - amount - service_charge
    print("\n=== ATM Transaction Receipt ===")
    print(f"Initial Balance:\t$", initial_balance)
    print(f"Withdrawal Amount:\t$", amount)
    print(f"Service Charge (2%):\t$", service_charge)

    print("=====")
    print(f"Final Balance:\t\t$", final_balance)
    print("=====")
    return final_balance

print("=====")
print("=== Bank ATM ===")
print("=====")
initial_balance = 10000
max_attempts = 3
attempt = 1
correct_pin = 1234
```

```

while attempt <= max_attempts:
    pin = int(input(f"Enter 4-digit PIN (Attempt {attempt}/3): "))

    if pin == correct_pin:
        print(f"Current Balance: ", initial_balance)
        amount = withdrawal(initial_balance)
        service_charge = service_charge(amount)
        print_receipt(initial_balance, amount, service_charge)
        print("Transaction completed successfully!")
        break
    else:
        print("Invalid PIN! Must be a 4-digit number.")
        attempt += 1

if attempt > max_attempts:
    print("\n=== Card Blocked ===")
    print("=====")

```

## Output:

```

=====
=== Bank ATM ===
=====
Enter 4-digit PIN (Attempt 1/3): 1234
Current Balance: 10000
Enter withdrawal amount: 500

=== ATM Transaction Receipt ===
Initial Balance:      $ 10000
Withdrawal Amount:    $ 500.0
Service Charge (2%):  $ 10.0
=====
Final Balance:        $ 9490.0
=====
Transaction completed successfully!

```



## Program 2: Grocery Store Inventory System

```
inventory = {
    "apple": {"stock": 50, "price": 0.5},
    "banana": {"stock": 100, "price": 0.3},
    "orange": {"stock": 75, "price": 0.4},
    "milk": {"stock": 20, "price": 2.5},
    "bread": {"stock": 30, "price": 1.5},
}

def discountfn(total):
    if total >= 50:
        return 0.20
    elif total >= 30:
        return 0.15
    elif total >= 15:
        return 0.10
    return 0.0

def process_purchase():
    cart = {}
    while True:
        print("\nAvailable Items:")
        for item, details in inventory.items():
            print(f"{item}: ${details['price']}, Stock: {details['stock']}")

        item = input("\nEnter item name (or 'done' to finish): ").lower()
        if item == "done":
            break
        if item not in inventory:
            print("Item not found!")
            continue

        quantity = int(input(f"Enter quantity for {item}: "))
        if quantity <= 0:
            print("Quantity must be positive!")
            continue
        if quantity > inventory[item]["stock"]:
            print(f"Not enough stock! Available: {inventory[item]['stock']}")
            continue

        cart[item] = cart.get(item, 0) + quantity
        inventory[item]["stock"] -= quantity
        print(f"Added {quantity} {item}(s) to cart.")

    return cart

def print_bill(cart):
    if not cart:
        print("\nCart is empty!")
        return

    subtotal = 0
    print("\n=== Final Bill ===")
    print("Item\t\tQuantity\tPrice\tTotal")
    print("=====")
```

```

for item, quantity in cart.items():
    price = inventory[item]["price"]
    total = price * quantity
    subtotal += total
    print(f"{item:<15}\t{quantity}\t\t${price:.2f}\t${total:.2f}")

discount_rate = discountfn(subtotal)
discount = subtotal * discount_rate
after_discount = subtotal - discount
tax = after_discount * 0.05
total = after_discount + tax

print("=====")
print(f"Subtotal:\t\t\t${subtotal:.2f}")
if discount > 0:
    print(f"Discount ({discount_rate * 100:.0f}%):\t\t\t-${discount:.2f}")
print(f"After Discount:\t\t\t${after_discount:.2f}")
print(f"Tax (5%):\t\t\t${tax:.2f}")
print(f"Total:\t\t\t\t${total:.2f}")
print("=====")

print("=====")
print("=== Welcome to the Grocery Store! ===")
print("=====")
cart = process_purchase( )
print_bill(cart)

```

## Output:

```

=====
=== Welcome to the Grocery Store! ===
=====

Available Items:
apple: $0.5, Stock: 50
banana: $0.3, Stock: 100
orange: $0.4, Stock: 75
milk: $2.5, Stock: 20
bread: $1.5, Stock: 30

Enter item name (or 'done' to finish): apple
Enter quantity for apple: 5
Added 5 apple(s) to cart.

Available Items:
apple: $0.5, Stock: 45
banana: $0.3, Stock: 100
orange: $0.4, Stock: 75
milk: $2.5, Stock: 20
bread: $1.5, Stock: 30

Enter item name (or 'done' to finish): done

=== Final Bill ===

```

| Item            | Quantity | Price  | Total  |
|-----------------|----------|--------|--------|
| apple           | 5        | \$0.50 | \$2.50 |
| Subtotal:       |          | \$2.50 |        |
| After Discount: |          | \$2.50 |        |
| Tax (5%):       |          | \$0.12 |        |
| Total:          |          | \$2.62 |        |

```

=====

```

### Program 3: Student Grading with Subject Weightage

```
WEIGHTAGES = (0.30, 0.25, 0.20, 0.15, 0.10)
SUBJECTS = ('Math', 'Physics', 'Chemistry', 'English', 'Computer Science')

def input_marks():
    marks = []
    for subject in SUBJECTS:
        while True:
            mark = float(input(f"Enter marks for {subject} (0-100): "))
            if 0 <= mark <= 100:
                marks.append(mark)
                break
            else:
                print("Marks must be between 0 and 100!")
    return marks

def weighted_average(marks):
    weighted_sum = sum(mark * weight for mark, weight in zip(marks, WEIGHTAGES))
    return weighted_sum

def assign_grade(weighted_avg):
    if weighted_avg >= 90:
        return 'A'
    elif weighted_avg >= 80:
        return 'B'
    elif weighted_avg >= 70:
        return 'C'
    elif weighted_avg >= 60:
        return 'D'
    else:
        return 'Fail'

def check_scholarship(weighted_avg):
    return weighted_avg >= 80

def display_results(marks, weighted_avg, grade, scholarship):
    print("\n=== Student Grade Report ===")
    print("Subject\t\t\tMarks")
    print("-" * 25)
    for subject, mark in zip(SUBJECTS, marks):
        print(f"{subject:<15}\t", mark)
    print("-" * 25)
    print(f"Weighted Average:\t", weighted_avg)
    print(f"Grade:\t\t\t", grade)
    print(f"Scholarship Eligible:\t{'Yes' if scholarship else 'No'}")
    print("=====")

print("=====")
print("=== Student Grading System ===")
print("=====")
marks = input_marks()
weighted_avg = weighted_average(marks)
grade = assign_grade(weighted_avg)
scholarship = check_scholarship(weighted_avg)
display_results(marks, weighted_avg, grade, scholarship)
```

## Output:

```
=====
=== Student Grading System ===
=====
Enter marks for Math (0-100): 70
Enter marks for Physics (0-100): 80
Enter marks for Chemistry (0-100): 79
Enter marks for English (0-100): 93
Enter marks for Computer Science (0-100): 95

=== Student Grade Report ===
Subject          Marks
-----
Math              70.0
Physics           80.0
Chemistry         79.0
English           93.0
Computer Science      95.0
-----
Weighted Average:      80.25
Grade:                B
Scholarship Eligible:  Yes
=====
```

## Program 4: BMI + Health Recommendation

```
def get_user_input():
    while True:
        weight = float(input("Enter weight in kilograms: "))
        if weight <= 0:
            print("Weight must be positive!")
            continue
        height = float(input("Enter height in meters: "))
        if height <= 0:
            print("Height must be positive!")
            continue
        age = int(input("Enter age in years: "))
        if age <= 0:
            print("Age must be positive!")
            continue
        return weight, height, age

def calculate_bmi(weight, height):
    return weight / (height ** 2)

def bmi_category(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif 18.5 <= bmi < 25:
        return "Normal"
    elif 25 <= bmi < 30:
        return "Overweight"
    else:
        return "Obese"

def health_recommendations(bmi_category, age):
    recommendations = {
        "Underweight": {
            "diet": """"Increase caloric intake with nutrient-dense foods like nuts,
            whole grains, and healthy fats. Consider protein-rich meals.""",
            "exercise": """"Focus on strength training to build muscle mass
            (e.g., 2-3 sessions/week). Avoid excessive cardio.""",
        },
        "Normal": {
            "diet": """"Maintain a balanced diet with vegetables, lean proteins,
            and whole grains. Monitor portion sizes.""",
            "exercise": """"Engage in regular exercise
            (e.g., 150 min/week of moderate cardio + 2 strength sessions).""",
        },
        "Overweight": {
            "diet": """"Reduce calorie intake with a focus on vegetables,
            lean proteins, and low-sugar foods. Limit processed carbs.""",
            "exercise": """"Combine cardio (e.g., 200 min/week) with strength training
            (2-3 sessions/week).""",
        },
        "Obese": {
            "diet": """"Adopt a low-calorie, nutrient-rich diet. Consult a dietitian
            for a personalized plan. Avoid sugary drinks.""",
            "exercise": """"Start with low-impact activities like walking or swimming
            (30 min/day, 5 days/week). Gradually add strength training.""",
        }
    }
```

```

age_modifier = ""
if age < 18:
    age_modifier = """Consult a pediatrician before making significant diet
    or exercise changes."""
elif age >= 65:
    age_modifier = """Consider low-impact exercises and consult a doctor
    for diet changes."""

rec = recommendations[bmi_category]
rec["age_modifier"] = age_modifier
return rec

def display_results(weight, height, age, bmi, bmi_category, recommendations):
    print("\n=== BMI and Health Report ===")
    print(f"Weight: {weight} kg")
    print(f"Height: {height} m")
    print(f"Age: {age} years")
    print(f"BMI: {bmi}")
    print(f"Category: {bmi_category}")
    print("\nHealth Recommendations:")
    print(f"Diet: {recommendations['diet']}")
    print(f"Exercise: {recommendations['exercise']}")
    if recommendations['age_modifier']:
        print(f>Note: {recommendations['age_modifier']}")
    print("=====")

print("=====")
print("=== BMI Calculator ===")
print("=====")
weight, height, age = get_user_input()
bmi = calculate_bmi(weight, height)
bmi_category = bmi_category(bmi)
recommendations = health_recommendations(bmi_category, age)
display_results(weight, height, age, bmi, bmi_category, recommendations)

```

## Output:

```

=====
=== BMI Calculator ===
=====
Enter weight in kilograms: 78
Enter height in meters: 2.5
Enter age in years: 20

=== BMI and Health Report ===
Weight: 78.0 kg
Height: 2.5 m
Age: 20 years
BMI: 12.48
Category: Underweight

Health Recommendations:
Diet: Increase caloric intake with nutrient-dense foods like nuts,
      whole grains, and healthy fats. Consider protein-rich meals.
Exercise: Focus on strength training to build muscle mass
          (e.g., 2-3 sessions/week). Avoid excessive cardio.
=====

```

## Program 5: Telecom Prepaid Recharge System

```
RATES = { 'call': 1.5, 'sms': 0.2, 'data': 0.5}

def initial_balance():
    while True:
        balance = float(input("Enter initial balance: "))
        if balance >= 0:
            return balance
        print("Balance cannot be negative!")

def process_recharge(current_balance):
    while True:
        recharge = float(input("Enter recharge amount (0 to skip): "))
        if recharge < 0:
            print("Recharge amount cannot be negative!")
            continue
        bonus = 0.1 * recharge if recharge > 1000 else 0
        return current_balance + recharge + bonus, recharge > 0

def process_usage(balance):
    usage_log = {'call': 0, 'sms': 0, 'data': 0}
    while True:
        print("\nAvailable Services: call (per min), sms, data (per MB)")
        print(f"Current Balance: $", balance)
        service = input("Enter service (call/sms/data) or 'done' to finish: ").lower()
        if service == 'done':
            break
        if service not in RATES:
            print("Invalid service!")
            continue

        amount = float(input(f"Enter {service} usage ({'min' if service == 'call' else
'MB' if service == 'data' else 'SMS'}): "))
        if amount < 0:
            print("Usage cannot be negative!")
            continue
        cost = amount * RATES[service]
        if cost > balance:
            print(f"Insufficient balance! Cost: ${cost}, Available: ${balance}")
            continue
        balance -= cost
        usage_log[service] += amount
        print(f"Used {amount} {service}. Cost: ${cost}, New Balance: ${balance}")

    return balance, usage_log

def print_bill(initial_balance, final_balance, usage_log, recharged):
    print("\n== Usage Summary ==")
    print("Service\t\tUsage\t\tCost")
    print("-" * 35)
    total_cost = 0
    for service, amount in usage_log.items():
        if amount > 0:
            cost = amount * RATES[service]
            total_cost += cost
            unit = 'min' if service == 'call' else 'MB' if service == 'data' else 'SMS'
            print(f"{service.capitalize():<15}\t{amount} {unit}\t${cost}")
    print("-" * 35)
    print(f"Initial Balance:\t$", initial_balance)
```

```

    if recharged:
        print(f"Recharge Applied")
    print(f"Total Cost:\t\t$", total_cost)
    print(f"Final Balance:\t\t$", final_balance)
    if final_balance < 0.2 * initial_balance:
        print("Warning: Balance below 20% of initial! Please recharge soon.")
    print("=====")

print("=====")
print("=== Telecom Prepaid Recharge System ===")
print("=====")
balance = initial_balance()
initial_balance = balance
recharged = False

while True:
    balance, recharged = process_recharge(balance)
    balance, usage_log = process_usage(balance)
    print_bill(initial_balance, balance, usage_log, recharged)

    cont = input("\nContinue with more usage? (yes/no): ").lower()
    if cont != 'yes':
        break

```

## Output:

```

=====
=== Telecom Prepaid Recharge System ===
=====
Enter initial balance: 100
Enter recharge amount (0 to skip): 50

Available Services: call (per min), sms, data (per MB)
Current Balance: $ 150.0
Enter service (call/sms/data) or 'done' to finish: done

=== Usage Summary ===
Service          Usage          Cost
-----
-----
Initial Balance:      $ 100.0
Recharge Applied
Total Cost:           $ 0
Final Balance:        $ 150.0
=====

Continue with more usage? (yes/no): no

```



## Program 6: E-Commerce Discount Engine with Membership Levels

```
MEMBERSHIP_DISCOUNTS = {'regular': 0.0, 'gold': 0.10, 'platinum': 0.15}

def user_input():
    while True:
        bill_amount = float(input("Enter bill amount: "))
        if bill_amount <= 0:
            print("Bill amount must be positive!")
            continue
        membership = input("Enter membership type (Regular/Gold/Platinum): ").lower()
        if membership not in MEMBERSHIP_DISCOUNTS:
            print("Invalid membership type! Choose Regular, Gold, or Platinum.")
            continue
        return bill_amount, membership

def calculate_discounts(bill_amount, membership):
    membership_discount_rate = MEMBERSHIP_DISCOUNTS[membership]
    membership_discount = bill_amount * membership_discount_rate

    flat_discount = 0.05 * bill_amount if bill_amount > 10000 else 0

    total_discount = membership_discount + flat_discount
    discounted_amount = bill_amount - total_discount

    tax = discounted_amount * 0.05
    final_amount = discounted_amount + tax

    return membership_discount, flat_discount, tax, final_amount

def print_receipt(bill_amount, membership, membership_discount, flat_discount, tax,
final_amount):
    print("\n=== Itemized Receipt ===")
    print(f"Original Bill Amount:\t$", bill_amount)
    print(f"Membership Type:\t\t", membership.capitalize())
    print("-" * 30)
    if membership_discount > 0:
        print(f"Membership Discount ({MEMBERSHIP_DISCOUNTS[membership]*100}%):\t-$",
membership_discount)
    if flat_discount > 0:
        print(f"High Bill Discount (5%):\t-$", flat_discount)
    print(f"Subtotal after Discounts:\t$", bill_amount - membership_discount - flat_discount)
    print(f"Tax (5%):\t\t$", tax)
    print("-" * 30)
    print(f"Final Amount:\t\t$", final_amount)
    print("=====")

print("=====")
print("=== E-Commerce Discount Engine ===")
print("=====")
bill_amount, membership = user_input()
membership_discount, flat_discount, tax, final_amount = calculate_discounts(bill_amount,
membership)
print_receipt(bill_amount, membership, membership_discount, flat_discount, tax, final_amount)
```

## Output:

```
=====
=== E-Commerce Discount Engine ===
=====
Enter bill amount: 500
Enter membership type (Regular/Gold/Platinum): Gold

=== Itemized Receipt ===
Original Bill Amount:   $ 500.0
Membership Type:                Gold
-----
Membership Discount (10.0%):    -$ 50.0
Subtotal after Discounts:        $ 450.0
Tax (5%):                      $ 22.5
-----
Final Amount:                  $ 472.5
=====
```

## Program 7: Hospital Emergency Unit with Multi-Condition Triage

```
def patient_details():
    while True:
        temperature = float(input("Enter temperature (°C, e.g., 36.5): "))
        if not 34 <= temperature <= 42:
            print("Temperature must be between 34°C and 42°C!")
            continue
        pulse = int(input("Enter pulse rate (beats/min, e.g., 70): "))
        if not 40 <= pulse <= 140:
            print("Pulse rate must be between 40 and 140!")
            continue
        oxygen = int(input("Enter oxygen saturation (% , e.g., 95): "))
        if not 60 <= oxygen <= 100:
            print("Oxygen saturation must be between 60% and 100%!")
            continue
        bp_systolic = int(input("Enter systolic blood pressure (mmHg, e.g., 120): "))
        if not 70 <= bp_systolic <= 200:
            print("Systolic BP must be between 70 and 200 mmHg!")
            continue
        return temperature, pulse, oxygen, bp_systolic

def classify_triage(temperature, pulse, oxygen, bp_systolic):
    if oxygen < 90 or bp_systolic < 90 or bp_systolic > 180:
        if oxygen < 85 or bp_systolic < 80 or bp_systolic > 200 or temperature > 40:
            return "Critical", "Admit in ICU"
        return "Urgent", "Immediate doctor attention"
    elif temperature > 38.5 or pulse > 100 or pulse < 50:
        if temperature > 39.5 or pulse > 120 or pulse < 45 or oxygen < 92:
            return "Urgent", "Immediate doctor attention"
        return "Observation", "Monitor closely and consult doctor"
    else:
        return "Stable", "Routine check-up recommended"

def display_triage_results(temperature, pulse, oxygen, bp_systolic, category, suggestion):
    print("\n=== Triage Report ===")
    print(f"Temperature: {temperature}°C")
    print(f"Pulse: {pulse} beats/min")
    print(f"Oxygen Saturation: {oxygen}%")
    print(f"Systolic BP: {bp_systolic} mmHg")
    print(f"Triage Category: ", category)
    print(f"Recommendation: ", suggestion)
    print("=====")

print("=====")
print("=== Hospital Emergency Unit Triage System ===")
print("=====")
temperature, pulse, oxygen, bp_systolic = patient_details()
category, suggestion = classify_triage(temperature, pulse, oxygen, bp_systolic)
display_triage_results(temperature, pulse, oxygen, bp_systolic, category, suggestion)
```

### Output:

```
=====
=== Hospital Emergency Unit Triage System ===
=====
Enter temperature (°C, e.g., 36.5): 39.9
Enter pulse rate (beats/min, e.g., 70): 76
Enter oxygen saturation (% , e.g., 95): 92
Enter systolic blood pressure (mmHg, e.g., 120): 120

=== Triage Report ===
Temperature: 39.9°C
Pulse: 76 beats/min
Oxygen Saturation: 92%
Systolic BP: 120 mmHg
Triage Category: Urgent
Recommendation: Immediate doctor attention
=====
```

## Program 8: Online Examination Grader with Negative Marking

```
def exam_details():
    while True:
        try:
            total_marks = float(input("Enter total marks: "))
            if total_marks <= 0:
                print("Total marks must be positive!")
                continue
            obtained_marks = float(input("Enter obtained marks: "))
            if obtained_marks < 0 or obtained_marks > total_marks:
                print(f"Obtained marks must be between 0 and {total_marks}!")
                continue
            wrong_answers = int(input("Enter number of wrong answers: "))
            if wrong_answers < 0:
                print("Wrong answers cannot be negative!")
                continue
            return total_marks, obtained_marks, wrong_answers
        except ValueError:
            print("Invalid input! Please enter numeric values.")

def calculate_final_marks(obtained_marks, wrong_answers):
    negative_marks = wrong_answers * 0.25
    final_marks = obtained_marks - negative_marks
    return max(final_marks, 0)

def calculate_percentage(final_marks, total_marks):
    return (final_marks / total_marks) * 100

def assign_division(percentage):
    if percentage >= 80:
        return "First Division"
    elif percentage >= 60:
        return "Second Division"
    elif percentage >= 40:
        return "Third Division"
    else:
        return "Fail"

def check_scholarship(percentage):
    return percentage >= 80

def suggestion(division):
    return "Repeat Exam" if division == "Fail" else "Continue Studies"

def display_results(total_marks, obtained_marks, wrong_answers, final_marks, percentage,
division, scholarship, suggestion):
    print("\n=== Exam Results ===")
    print(f"Total Marks:\t\t", total_marks)
    print(f"Obtained Marks:\t\t", obtained_marks)
    print(f"Wrong Answers:\t\t", wrong_answers)
    print(f"Negative Marking (0.25/wrong):\t", wrong_answers * 0.25)
    print(f"Final Marks:\t\t", final_marks)
    print(f"Percentage:\t\t{percentage}%")
    print(f"Division:\t\t", division)
    print(f"Scholarship Eligible:\t{'Yes' if scholarship else 'No'}")
    print(f"Suggestion:\t\t", suggestion)
    print("=====")
```

```

print("=====")
print("=== Online Examination Grader ===")
print("=====")
total_marks, obtained_marks, wrong_answers = exam_details()
final_marks = calculate_final_marks(obtained_marks, wrong_answers)
percentage = calculate_percentage(final_marks, total_marks)
division = assign_division(percentage)
scholarship = check_scholarship(percentage)
suggestion = suggestion(division)
display_results(total_marks, obtained_marks, wrong_answers, final_marks, percentage,
division, scholarship, suggestion)

```

## Output:

```

=====
=== Online Examination Grader ===
=====
Enter total marks: 500
Enter obtained marks: 395
Enter number of wrong answers: 18

=== Exam Results ===
Total Marks:           500.0
Obtained Marks:        395.0
Wrong Answers:         18
Negative Marking (0.25/wrong):  4.5
Final Marks:           390.5
Percentage:             78.10000000000001%
Division:               Second Division
Scholarship Eligible:  No
Suggestion:             Continue Studies
=====

```

## Program 9: Movie Ticket Booking with Age & Timing Policy

```
TICKET_PRICES = {'normal': 10.0, '3d': 12.5, 'imax': 15.0}

def user_input():
    while True:
        age = int(input("Enter age: "))
        if age <= 0:
            print("Age must be positive!")
            continue
        showtime = float(input("Enter showtime (24-hour format, e.g., 14.5 for 2:30 PM):"))
        if not 0 <= showtime <= 24:
            print("Showtime must be between 00:00 and 24:00!")
            continue
        ticket_type = input("Enter ticket type (Normal/3D/IMAX): ").lower()
        if ticket_type not in TICKET_PRICES:
            print("Invalid ticket type! Choose Normal, 3D, or IMAX.")
            continue
        return age, showtime, ticket_type

def apply_policies(age, showtime, ticket_type):
    base_price = TICKET_PRICES[ticket_type]
    restriction = ""
    discount = 0.0
    surcharge = 0.0

    if age < 12 and showtime > 20:
        restriction = "Children under 12 are not allowed for shows after 8:00 PM."
        return base_price, discount, surcharge, restriction

    if age >= 60:
        discount = base_price * 0.30

    if 18 <= showtime <= 22:
        surcharge = base_price * 0.10

    return base_price, discount, surcharge, restriction

def calculate_final_price(base_price, discount, surcharge):
    return base_price - discount + surcharge

def print_ticket(age, showtime, ticket_type, base_price, discount, surcharge, final_price, restriction):
    print("\n=== Movie Ticket Receipt ===")
    if restriction:
        print(f"Restriction: {restriction}")
        print("Ticket purchase denied.")
    else:
        print(f"Age: {age}")
        print(f"Showtime: {int(showtime):02d}:{int((showtime % 1) * 60):02d}")
        print(f"Ticket Type: ", ticket_type.capitalize())
        print("-" * 30)
        print(f"Base Price:\t\t${base_price:.2f}")
        if discount > 0:
            print(f"Senior Discount (30%):\t-$", discount)
        if surcharge > 0:
            print(f"Peak Hour Surcharge (10%):\t$", surcharge)
        print("-" * 30)
        print(f"Final Price:\t\t$", final_price)
    print("=====")
```

```

print("=====")
print("=== Movie Ticket Booking System ===")
print("=====")
age, showtime, ticket_type = user_input()
base_price, discount, surcharge, restriction = apply_policies(age, showtime, ticket_type)
final_price = calculate_final_price(base_price, discount, surcharge) if not restriction
else 0
print_ticket(age, showtime, ticket_type, base_price, discount, surcharge, final_price,
restriction)

```

## Output:

```

=====
=== Movie Ticket Booking System ===
=====
Enter age: 20
Enter showtime (24-hour format, e.g., 14.5 for 2:30 PM): 15.5
Enter ticket type (Normal/3D/IMAX): 3D

=== Movie Ticket Receipt ===
Age: 20
Showtime: 15:30
Ticket Type: 3d
-----
Base Price:                $12.50
-----
Final Price:                $ 12.5
=====

```

## Program 10: Weather-based Outfit + Travel Suggestion

```
WEATHER_TYPES = ['rainy', 'sunny', 'cold']
EVENT_TYPES = ['casual', 'business', 'party']

def user_input():
    while True:
        temperature = float(input("Enter temperature (°C): "))
        weather = input("Enter weather (Rainy/Sunny/Cold): ").lower()
        if weather not in WEATHER_TYPES:
            print("Invalid weather! Choose Rainy, Sunny, or Cold.")
            continue
        event = input("Enter event type (Casual/Business/Party): ").lower()
        if event not in EVENT_TYPES:
            print("Invalid event type! Choose Casual, Business, or Party.")
            continue
        return temperature, weather, event

def outfit_suggestion(temperature, weather, event):
    outfit = ""
    accessories = []

    if weather == 'rainy':
        accessories.append("umbrella")
    elif weather == 'cold':
        accessories.append("gloves")

    if event == 'casual':
        if weather == 'sunny' and temperature > 25:
            outfit = "t-shirt and shorts"
            accessories.append("sunglasses")
        elif weather == 'cold' or temperature < 15:
            outfit = "sweater and jeans"
            accessories.append("scarf")
        else:
            outfit = "light jacket and pants"
    elif event == 'business':
        if temperature > 35:
            outfit = "light formal suit"
        else:
            outfit = "formal suit and tie"
            accessories.append("briefcase")
    elif event == 'party':
        if temperature > 25:
            outfit = "stylish dress or shirt with slacks"
            accessories.append("sunglasses")
        else:
            outfit = "elegant jacket and dress pants"

    return outfit, accessories

def get_travel_suggestion(weather):
    if weather == 'rainy':
        return "Use public transport or a car to stay dry."
    elif weather == 'cold':
        return "Opt for a heated vehicle or short walks."
    else:
        return "Walking or biking is suitable."
```



```

def display_suggestions(temperature, weather, event, outfit, accessories, travel):
    print("\n=== Outfit and Travel Suggestions ===")
    print(f"Temperature: {temperature:.1f}°C")
    print(f"Weather: ", weather.capitalize())
    print(f"Event: ", event.capitalize())
    print("-" * 35)
    print(f"Outfit: ", outfit)
    print(f"Accessories: {'', '.join(accessories) if accessories else 'None'}")
    print(f"Travel: ", travel)
    print("=====")

print("=====")
print("=== Weather-based Outfit and Travel Suggestion System ===")
print("=====")
temperature, weather, event = user_input()
outfit, accessories = outfit_suggestion(temperature, weather, event)
travel = get_travel_suggestion(weather)
display_suggestions(temperature, weather, event, outfit, accessories, travel)

```

## Output:

```

=====
=== Weather-based Outfit and Travel Suggestion System ===
=====
Enter temperature (°C): 39.9
Enter weather (Rainy/Sunny/Cold): Sunny
Enter event type (Casual/Business/Party): Party

=== Outfit and Travel Suggestions ===
Temperature: 39.9°C
Weather: Sunny
Event: Party
-----
Outfit: stylish dress or shirt with slacks
Accessories: sunglasses
Travel: Walking or biking is suitable.
=====

```

## Program 11: ATM Withdrawal with Note Counting + Receipt

```
DENOMINATIONS = [1000, 500, 100, 50, 10, 1]

def user_input(initial_balance):
    while True:
        amount = int(input("Enter withdrawal amount: "))
        if amount <= 0:
            print("Amount must be positive!")
            continue
        if amount > initial_balance:
            print(f"Insufficient balance! Available: ${initial_balance:.2f}")
            continue
        return amount

def calculate_notes(amount):
    notes = {}
    remaining = amount
    for denom in DENOMINATIONS:
        notes[denom] = remaining // denom
        remaining %= denom
    return notes

def calculate_service_charge(amount):
    return amount * 0.005

def print_receipt(initial_balance, amount, notes, service_charge):
    final_balance = initial_balance - amount - service_charge
    print("\n=== ATM Withdrawal Receipt ===")
    print(f"Initial Balance:\t$", initial_balance)
    print(f"Withdrawal Amount:\t$", amount)
    print("-" * 30)
    print("Note Breakdown:")
    for denom, count in notes.items():
        if count > 0:
            print(f"${denom} x {count}")
    print("-" * 30)
    print(f"Service Charge (0.5%):\t$", service_charge)
    print(f"Final Balance:\t\t$", final_balance)
    print("=====")

print("=====")
print("=== ATM Withdrawal System ===")
print("=====")
initial_balance = 10000
amount = user_input(initial_balance)
notes = calculate_notes(amount)
service_charge = calculate_service_charge(amount)
print_receipt(initial_balance, amount, notes, service_charge)
```

## Output:

```
=====
=== ATM Withdrawal System ===
=====
Enter withdrawal amount: 5000

=== ATM Withdrawal Receipt ===
Initial Balance:      $ 10000
Withdrawal Amount:    $ 5000
-----
Note Breakdown:
$1000 x 5
-----
Service Charge (0.5%): $ 25.0
Final Balance:        $ 4975.0
=====
```

## Program 12: Library Fine Calculator with Membership Levels

```
FINE_RATES = {'regular': [10, 20, 50], 'premium': [5, 10, 25]}

def user_input():
    while True:
        days_late = int(input("Enter days late: "))
        if days_late < 0:
            print("Days late cannot be negative!")
            continue
        membership = input("Enter membership type (Regular/Premium): ").lower()
        if membership not in FINE_RATES:
            print("Invalid membership! Choose Regular or Premium.")
            continue
        return days_late, membership

def calculate_fine(days_late, membership):
    rates = FINE_RATES[membership]
    total_fine = 0
    breakdown = []

    if days_late == 0:
        return 0, breakdown

    days1 = min(10, days_late)
    if days1 > 0:
        fine1 = days1 * rates[0]
        total_fine += fine1
        breakdown.append(f"Days 1-10: {days1} days ${rates[0]} = ${fine1}")

    days2 = min(10, max(0, days_late - 10))
    if days2 > 0:
        fine2 = days2 * rates[1]
        total_fine += fine2
        breakdown.append(f"Days 11-20: {days2} days ${rates[1]} = ${fine2}")

    days3 = max(0, days_late - 20)
    if days3 > 0:
        fine3 = days3 * rates[2]
        total_fine += fine3
        breakdown.append(f"Days 21+: {days3} days ${rates[2]} = ${fine3}")

    return total_fine, breakdown

def check_membership_status(days_late):
    if days_late > 30:
        return "Membership Cancelled"
    return "Active"

def print_fine_slip(days_late, membership, total_fine, breakdown, status):
    print("\n=== Library Fine Slip ===")
    print(f"Days Late: ", days_late)
    print(f"Membership: ", membership.capitalize())
    print("-" * 30)
```

```

    if breakdown:
        print("Fine Breakdown:")
        for line in breakdown:
            print(f" {line}")
        print("-" * 30)
        print(f"Total Fine: $", total_fine)
    else:
        print("No fine applicable.")
    print(f"Membership Status: ", status)
    if status == "Membership Cancelled":
        print("Please contact library administration immediately.")
    print("=====")

print("=====")
print("=== Library Fine Calculator ===")
print("=====")
days_late, membership = user_input()
total_fine, breakdown = calculate_fine(days_late, membership)
status = check_membership_status(days_late)
print_fine_slip(days_late, membership, total_fine, breakdown, status)

```

## Output:

```

=====
=== Library Fine Calculator ===
=====
Enter days late: 9
Enter membership type (Regular/Premium): Premium

=== Library Fine Slip ===
Days Late: 9
Membership: Premium
-----
Fine Breakdown:
  Days 1-10: 9 days $5 = $45
-----
Total Fine: $ 45
Membership Status:  Active
=====

```

## Program 13: Sales Analysis Dashboard

```
MONTHS = ["January", "February", "March", "April", "May", "June",
          "July", "August", "September", "October", "November", "December"]

def sales_data():
    sales = []
    print("Enter sales for each month:")
    for month in MONTHS:
        while True:
            try:
                sale = float(input(f"{month}: $"))
                if sale < 0:
                    print("Sales cannot be negative!")
                    continue
                sales.append(sale)
                break
            except ValueError:
                print("Invalid input! Please enter a numeric value.")
    return sales

def analyze_sales(sales):
    max_sale = max(sales)
    min_sale = min(sales)
    avg_sale = sum(sales) / len(sales)
    return max_sale, min_sale, avg_sale

def print_analysis(sales, max_sale, min_sale, avg_sale):
    print("\n=== Sales Analysis Dashboard ===")
    print(f"Maximum Sales: ${max_sale} ({MONTHS[sales.index(max_sale)]})")
    print(f"Minimum Sales: ${min_sale} ({MONTHS[sales.index(min_sale)]})")
    print(f"Average Sales: ${avg_sale}")

    print("\nMonths Above Average:")
    above_avg = False
    for i, sale in enumerate(sales):
        if sale > avg_sale:
            print(f"{MONTHS[i]}: ${sale}")
            above_avg = True
    if not above_avg:
        print("No months above average.")

    print("\nMonthly Trend Analysis:")
    for i in range(1, len(sales)):
        trend = "Growth" if sales[i] > sales[i-1] else "Decline"
        print(f"{MONTHS[i]} vs {MONTHS[i-1]}: {trend} (${sales[i]} vs ${sales[i-1]})")
    print("=====")

    print("=====")
    print("=== Sales Analysis Dashboard ===")
    print("=====")
    sales = sales_data()
    max_sale, min_sale, avg_sale = analyze_sales(sales)
    print_analysis(sales, max_sale, min_sale, avg_sale)
```

## Output:

```
=====
=== Sales Analysis Dashboard ===
=====
Enter sales for each month:
January: $60
February: $70
March: $56
April: $45
May: $89
June: $78
July: $67
August: $46
September: $96
October: $89
November: $76
December: $67

=== Sales Analysis Dashboard ===
Maximum Sales: $96.0 (September)
Minimum Sales: $45.0 (April)
Average Sales: $69.91666666666667

Months Above Average:
February: $70.0
May: $89.0
June: $78.0
September: $96.0
October: $89.0
November: $76.0

Monthly Trend Analysis:
February vs January: Growth ($70.0 vs $60.0)
=====
March vs February: Decline ($56.0 vs $70.0)
=====
April vs March: Decline ($45.0 vs $56.0)
=====
May vs April: Growth ($89.0 vs $45.0)
=====
June vs May: Decline ($78.0 vs $89.0)
=====
July vs June: Decline ($67.0 vs $78.0)
=====
August vs July: Decline ($46.0 vs $67.0)
=====
September vs August: Growth ($96.0 vs $46.0)
=====
October vs September: Decline ($89.0 vs $96.0)
=====
November vs October: Decline ($76.0 vs $89.0)
=====
December vs November: Decline ($67.0 vs $76.0)
=====
```

## Program 14: Multiplication Puzzle with Random Blanks

```
import random

def generate_table():
    table = [[i * j for j in range(1, 11)] for i in range(1, 11)]
    return table

def select_random_blanks(table):
    blanks = []
    while len(blanks) < 3:
        row = random.randint(0, 9)
        col = random.randint(0, 9)
        if (row, col) not in blanks:
            blanks.append((row, col))
    return blanks

def display_table(table, blanks):
    print("\nMultiplication Table (1-10):")
    print(" ", end="")
    for j in range(1, 11):
        print(f"{j:4}", end="")
    print("\n" + "-" * 44)
    for i in range(10):
        print(f"{i+1:2} |", end="")
        for j in range(10):
            if (i, j) in blanks:
                print(f"'?:>4", end="")
            else:
                print(f"{table[i][j]:>4}", end="")
        print()

def user_guesses(table, blanks):
    score = 0
    for row, col in blanks:
        correct_answer = table[row][col]
        while True:
            guess = int(input(f"\nGuess the value for {row+1} x {col+1}: "))
            if guess == correct_answer:
                print("Correct!")
                score += 1
            else:
                print(f"Wrong! Correct answer is {correct_answer}.")
                break
    return score

def performance_remark(score):
    if score == 3:
        return "Excellent"
    elif score == 2:
        return "Good"
    else:
        return "Try Again"
```



```

def print_result(score, remark):
    print("\n=== Puzzle Result ===")
    print(f"Score: {score}/3")
    print(f"Performance: {remark}")
    print("=====")

print("=====")
print("=== Multiplication Puzzle ===")
print("=====")
table = generate_table()
blanks = select_random_blanks(table)
display_table(table, blanks)
score = user_guesses(table, blanks)
remark = performance_remark(score)
print_result(score, remark)

```

## Output:

```

=====
=== Multiplication Puzzle ===
=====

Multiplication Table (1-10):
      1  2  3  4  5  6  7  8  9 10
-----
1 |  1  2  3  4  5  6  7  8  9 10
2 |  2  4  6  8 10 12 14 16 18 20
3 |  3  6  9 12 15 18 21 24 27 30
4 |  4  8 12 16 20 24 28 32 36 40
5 |  5 10 15 20 25 30 35 40 45 50
6 |  6 12 18 24 30 36 42 48 54 60
7 |  7 14 21 28 35 42 ?? 56 63 70
8 |  8 16 24 32 ?? 48 56 64 72 80
9 |  9 18 27 36 45 54 63 72 81 90
10 | 10 20 30 40 50 60 70 ?? 90 100

Guess the value for 10 x 8: 80
Correct!

Guess the value for 7 x 7: 49
Correct!

Guess the value for 8 x 5: 40
Correct!

=== Puzzle Result ===
Score: 3/3
Performance: Excellent
=====

```

## Program 15: Password Strength + Re-entry Attempts

```
def is_strong_password(password):
    length = len(password) >= 8

    digit = any(c.isdigit() for c in password)

    uppercase = any(c.isupper() for c in password)

    lowercase = any(c.islower() for c in password)

    special_chars = "!@#$%^&*(),.?\\\":{}|<>"
    special = any(c in special_chars for c in password)

    return length and digit and uppercase and lowercase and special

def check_password_strength(password, attempt):
    if is_strong_password(password):
        return True, "Password is strong!"
    else:
        weaknesses = []
        if len(password) < 8:
            weaknesses.append("at least 8 characters")
        if not any(c.isdigit() for c in password):
            weaknesses.append("at least one digit")
        if not any(c.isupper() for c in password):
            weaknesses.append("at least one uppercase letter")
        if not any(c.islower() for c in password):
            weaknesses.append("at least one lowercase letter")
        special_chars = "!@#$%^&*(),.?\\\":{}|<>"
        if not any(c in special_chars for c in password):
            weaknesses.append("at least one special character")
        return False, f"Password is weak! Missing: {'', ' '.join(weaknesses)}."
Attempt {attempt}/3."

print("=====")
print("=== Password Strength Checker ===")
print("=====")
max_attempts = 3
attempt = 1

while attempt <= max_attempts:
    password = input(f"\nEnter password (Attempt {attempt}/{max_attempts}): ")
    is_strong, message = check_password_strength(password, attempt)
    print(message)

    if is_strong:
        print("Account setup successful!")
        break
    attempt += 1

if attempt > max_attempts and not is_strong:
    print("\n=== Account Locked ===")
    print("Too many weak password attempts. Account locked.")
    print("=====")
```

## Output:

```
=====
=== Password Strength Checker ===
=====

Enter password (Attempt 1/3): tayyab@12345
Password is weak! Missing: at least one uppercase letter. Attempt 1/3.

Enter password (Attempt 2/3): Tayyab@12345
Password is strong!
Account setup successful!
```

## Program 16: Electricity Bill with Slabs & Penalty

```
def calculate_bill(units, late_payment=False):
    bill = 0
    breakdown = {}

    if units <= 100:
        bill = units * 5
        breakdown['0-100 units @ Rs 5/unit'] = units * 5
    elif units <= 200:
        bill = (100 * 5) + ((units - 100) * 7)
        breakdown['0-100 units @ Rs 5/unit'] = 100 * 5
        breakdown['101-200 units @ Rs 7/unit'] = (units - 100) * 7
    else:
        bill = (100 * 5) + (100 * 7) + ((units - 200) * 10)
        breakdown['0-100 units @ Rs 5/unit'] = 100 * 5
        breakdown['101-200 units @ Rs 7/unit'] = 100 * 7
        breakdown['>200 units @ Rs 10/unit'] = (units - 200) * 10

    penalty = bill * 0.15 if late_payment else 0
    total = bill + penalty

    return bill, penalty, total, breakdown

def print_bill(units, bill, penalty, total, breakdown):
    print("\n=== Electricity Bill Breakdown ===")
    print(f"Units Consumed: {units}")
    print("-" * 35)
    for slab, amount in breakdown.items():
        if amount > 0:
            print(f"{slab}: Rs {amount:.2f}")
    print("-" * 35)
    print(f"Subtotal: Rs {bill:.2f}")
    if penalty > 0:
        print(f"Late Payment Penalty (15%): Rs {penalty:.2f}")
    print(f"Total Amount: Rs {total:.2f}")
    print("=====")

print("=====")
print("=== Electricity Bill Calculator ===")
print("=====")
while True:
    units = float(input("Enter units consumed: "))
    if units < 0:
        print("Units cannot be negative!")
        continue
    late_payment = input("Late payment? (yes/no): ").lower() == 'yes'
    bill, penalty, total, breakdown = calculate_bill(units, late_payment)
    print_bill(units, bill, penalty, total, breakdown)
    break
```

## Output:

```
=====
=== Electricity Bill Calculator ===
=====
Enter units consumed: 200
Late payment? (yes/no): no

=== Electricity Bill Breakdown ===
Units Consumed: 200.0
-----
0-100 units @ Rs 5/unit: Rs 500.00
101-200 units @ Rs 7/unit: Rs 700.00
-----
Subtotal: Rs 1200.00
Total Amount: Rs 1200.00
=====
```

## Program 17: Cricket Match Predictor with Multiple Conditions

```
def predict_score(runs, balls, wickets, overs_left):
    if balls == 0:
        overs_played = 0
        run_rate = 0
        strike_rate = 0
    else:
        overs_played = balls / 6.0
        run_rate = runs / overs_played if overs_played > 0 else 0
        strike_rate = (runs / balls) * 100

    if run_rate < 5 and wickets > 6:
        prediction = "Losing."
    elif run_rate > 7 and wickets < 5:
        prediction = "Winning."
    else:
        prediction = "Balanced."
    return strike_rate, run_rate, prediction, overs_played

def user_input():
    while True:
        runs = int(input("Enter current runs: "))
        if runs < 0:
            print("Runs cannot be negative!")
            continue
        balls = int(input("Enter balls bowled: "))
        if balls < 0:
            print("Balls cannot be negative!")
            continue
        wickets = int(input("Enter wickets lost: "))
        if wickets < 0 or wickets > 10:
            print("Wickets must be between 0 and 10!")
            continue
        overs_left = float(input("Enter overs left: "))
        if overs_left < 0:
            print("Overs left cannot be negative!")
            continue
        return runs, balls, wickets, overs_left

def display_results(runs, balls, wickets, overs_left, strike_rate, run_rate,
prediction, overs_played):
    print("\n=== Cricket Match Prediction ===")
    print(f"Current Runs: ", runs)
    print(f"Balls Bowled: ", balls)
    print(f"Wickets Lost: ", wickets)
    print(f"Overs Left: ", overs_left)
    print(f"Overs Played: ", overs_played)
    print(f"Strike Rate: ", strike_rate)
    print(f"Run Rate: ", run_rate)
    print(f"Prediction: ", prediction)
    print("=====")

print("=====")
print("=== Cricket Match Predictor ===")
print("=====")
runs, balls, wickets, overs_left = user_input()
strike_rate, run_rate, prediction, overs_played = predict_score(runs, balls,
wickets, overs_left)
display_results(runs, balls, wickets, overs_left, strike_rate, run_rate,
prediction, overs_played)
```

## Output:

```
=====
=== Cricket Match Predictor ===
=====
Enter current runs: 300
Enter balls bowled: 300
Enter wickets lost: 9
Enter overs left: 2

=== Cricket Match Prediction ===
Current Runs: 300
Balls Bowled: 300
Wickets Lost: 9
Overs Left: 2.0
Overs Played: 50.0
Strike Rate: 100.0
Run Rate: 6.0
Prediction: Balanced.
=====
```

## Program 18: Hotel Booking with Tax & Discount

```
ROOM_RATES = {'standard': 2000, 'deluxe': 4000, 'suite': 6000}

def book_room(room_type, days, is_member):
    room_type = room_type.lower()
    if room_type not in ROOM_RATES:
        return 0, 0, 0, {}, f"Invalid room type! Choose from {'',
        '.join(ROOM_RATES.keys())}.'"

    base_cost = ROOM_RATES[room_type] * days
    breakdown = {f"{room_type.capitalize()} Room ({days} days @ Rs
    {ROOM_RATES[room_type]}/day)": base_cost}

    long_stay_discount = base_cost * 0.20 if days > 7 else 0
    member_discount = base_cost * 0.10 if is_member else 0
    total_discount = long_stay_discount + member_discount
    discounted_cost = base_cost - total_discount

    tax = discounted_cost * 0.12
    total_cost = discounted_cost + tax

    if long_stay_discount > 0:
        breakdown["Long Stay Discount (20%)"] = -long_stay_discount
    if member_discount > 0:
        breakdown["Member Discount (10%)"] = -member_discount
    breakdown["Tax (12%)"] = tax

    return base_cost, total_discount, tax, total_cost, breakdown, ""

def print_bill(room_type, days, is_member, base_cost, total_discount, tax,
total_cost, breakdown, error):
    print("\n=== Hotel Booking Bill Slip ===")
    if error:
        print(f"Error: ", error)
    else:
        print(f"Room Type: ", room_type.capitalize())
        print(f"Days: ", days)
        print(f"Membership Status: {'Member' if is_member else 'Non-Member'}")
        print("-" * 35)
        for item, amount in breakdown.items():
            print(f"{item}: Rs {amount}")
        print("-" * 35)
        print(f"Total Cost: Rs ", total_cost)
    print("=====")

def user_input():
    while True:
        room_type = input("Enter room type (Standard/Deluxe/Suite): ").lower()
        days = int(input("Enter number of days: "))
        if days <= 0:
            print("Days must be positive!")
            continue
        is_member = input("Is member? (yes/no): ").lower() == 'yes'
        return room_type, days, is_member

print("Hotel Booking System")
room_type, days, is_member = user_input()
base_cost, total_discount, tax, total_cost, breakdown, error =
book_room(room_type, days, is_member)
print_bill(room_type, days, is_member, base_cost, total_discount, tax,
total_cost, breakdown, error)
```



## Output:

```
Hotel Booking System
Enter room type (Standard/Deluxe/Suite): Suite
Enter number of days: 7
Is member? (yes/no): no

=== Hotel Booking Bill Slip ===
Room Type: Suite
Days: 7
Membership Status: Non-Member
-----
Suite Room (7 days @ Rs 6000/day): Rs 42000
Tax (12%): Rs 5040.0
-----
Total Cost: Rs 47040.0
=====
```

## Program 19: Flight Fare Calculator with International Tax

```
CLASS_RATES = {'economy': 10, 'business': 25, 'first': 40}

def calculate_fare(distance, class_type, international=False,
days_in_advance=0):
    class_type = class_type.lower()
    if class_type not in CLASS_RATES:
        return 0, 0, 0, {}, f"Invalid class type! Choose from {'',
'.join(CLASS_RATES.keys()).}"

    base_fare = distance * CLASS_RATES[class_type]
    breakdown = {f"{class_type.capitalize()} Class ({distance} units @ Rs
{CLASS_RATES[class_type]}/unit)": base_fare}

    fuel_surcharge = 500
    base_fare += fuel_surcharge
    breakdown["Fuel Surcharge"] = fuel_surcharge

    advance_discount = base_fare * 0.10 if days_in_advance > 30 else 0
    discounted_fare = base_fare - advance_discount
    if advance_discount > 0:
        breakdown["Advance Booking Discount (10%)"] = -advance_discount

    tax = discounted_fare * 0.18 if international else 0
    total_fare = discounted_fare + tax
    if tax > 0:
        breakdown["International Tax (18%)"] = tax

    return base_fare, advance_discount, tax, total_fare, breakdown, ""

def print_fare_breakdown(distance, class_type, international, days_in_advance,
base_fare, advance_discount, tax, total_fare, breakdown, error):
    print("\n=== Flight Fare Breakdown ===")
    if error:
        print(f"Error: {error}")
    else:
        print(f"Distance: {distance} units")
        print(f"Class Type: ", class_type.capitalize())
        print(f"International: {'Yes' if international else 'No'}")
        print(f"Days in Advance: ", days_in_advance)
        print("-" * 35)
        for item, amount in breakdown.items():
            print(f"{item}: Rs ", amount)
        print("-" * 35)
        print(f"Total Fare: Rs ", total_fare)
    print("=====")

def user_input():
    while True:
        distance = float(input("Enter distance (in units): "))
        if distance <= 0:
            print("Distance must be positive!")
            continue
        class_type = input("Enter class type (Economy/Business/First):
").lower()
        international = input("International flight? (yes/no): ").lower() ==
'yes'
        days_in_advance = int(input("Enter days in advance: "))
        if days_in_advance < 0:
            print("Days in advance cannot be negative!")
            continue
        return distance, class_type, international, days_in_advance
```

```

print("=====")
print("=== Flight Fare Calculator ===")
print("=====")
distance, class_type, international, days_in_advance = user_input()
base_fare, advance_discount, tax, total_fare, breakdown, error =
calculate_fare(distance, class_type, international, days_in_advance)
print_fare_breakdown(distance, class_type, international, days_in_advance,
base_fare, advance_discount, tax, total_fare, breakdown, error)

```

## Output:

```

=====
=== Flight Fare Calculator ===
=====
Enter distance (in units): 1000
Enter class type (Economy/Business/First): Business
International flight? (yes/no): no
Enter days in advance: 3

=== Flight Fare Breakdown ===
Distance: 1000.0 units
Class Type: Business
International: No
Days in Advance: 3
-----
Business Class (1000.0 units @ Rs 25/unit): Rs 25000.0
Fuel Surcharge: Rs 500
-----
Total Fare: Rs 25500.0
=====

```

## Program 20: Smart Vending Machine with Wallet System

```
ITEMS = {
    'cola': {'price': 2.50, 'stock': 10},
    'chips': {'price': 1.50, 'stock': 8},
    'candy': {'price': 1.00, 'stock': 12},
    'water': {'price': 1.25, 'stock': 0}
}

def vending_machine(item, amount_paid, wallet_balance):
    item = item.lower()
    breakdown = {}

    if item not in ITEMS:
        return 0, 0, wallet_balance, {}, f"Invalid item! Choose from {'',
        '.join(ITEMS.keys())}."

    if ITEMS[item]['stock'] == 0:
        alternatives = [k for k, v in ITEMS.items() if v['stock'] > 0]
        if alternatives:
            return 0, 0, wallet_balance, {}, f"{item.capitalize()} is out of
            stock. Try: {'', '.join(alternatives)}."
        return 0, 0, wallet_balance, {}, f"{item.capitalize()} is out of
        stock. No alternatives available."

    price = ITEMS[item]['price']
    if amount_paid < price:
        return 0, 0, wallet_balance, {}, f"Insufficient payment!
        {item.capitalize()} costs ${price}, paid ${amount_paid}."

    change = amount_paid - price
    wallet_balance = wallet_balance - price
    ITEMS[item]['stock'] -= 1
    breakdown[f"{item.capitalize()} Purchase"] = price
    if change > 0:
        breakdown["Change Returned"] = change

    return price, change, wallet_balance, breakdown, f"{item.capitalize()}
    dispensed successfully."

def print_receipt(item, amount_paid, price, change, wallet_balance, breakdown,
message):
    print("\n=== Vending Machine Receipt ===")
    print(f"Item: ", item.capitalize())
    print(f"Amount Paid: $", amount_paid)
    print("-" * 35)
    if breakdown:
        for desc, amount in breakdown.items():
            print(f"{desc}: ${amount}")
        print("-" * 35)
    print(f"New Wallet Balance: $", wallet_balance)
    print(f"Message: ", message)
    print("=====")
```

```

def user_input():
    while True:
        item = input("Enter item (Cola/Chips/Candy/Water): ").lower()
        amount_paid = float(input("Enter amount paid: $"))
        if amount_paid < 0:
            print("Amount paid cannot be negative!")
            continue
        wallet_balance = float(input("Enter current wallet balance: $"))
        if wallet_balance < 0:
            print("Wallet balance cannot be negative!")
            continue
        return item, amount_paid, wallet_balance

print("=====")
print("=== Vending Machine Simulator ===")
print("=====")
item, amount_paid, wallet_balance = user_input()
price, change, new_balance, breakdown, message = vending_machine(item,
amount_paid, wallet_balance)
print_receipt(item, amount_paid, price, change, new_balance, breakdown,
message)

```

## Output:

```

=====
=== Vending Machine Simulator ===
=====
Enter item (Cola/Chips/Candy/Water): Cola
Enter amount paid: $34
Enter current wallet balance: $50

=== Vending Machine Receipt ===
Item: Cola
Amount Paid: $ 34.0
-----
Cola Purchase: $2.5
Change Returned: $31.5
-----
New Wallet Balance: $ 47.5
Message: Cola dispensed successfully.
=====

```

---

## Lab 2

### “OOP”

#### Program 1: Simple Banking Management

```
class BankAccount:
    def __init__(self, acc_num, name, balance):
        self.acc_num = acc_num
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance!")
            return False
        else:
            self.balance = self.balance - amount
            return True

    def display_balance(self):
        print("\n=== Account Status ===")
        print(f"Account:\t{self.acc_num}")
        print(f"Holder:\t\t{self.name}")
        print("=====")
        print(f"Balance:\t${self.balance}")
        print("=====")

def print_receipt(type, amount, balance):
    print(f"\n=== {type} Receipt ===")
    print(f"Transaction:\t{type}")
    print(f"Amount:\t\t\t${amount}")
    print("=====")
    print(f"New Balance:\t${balance}")
    print("=====")

# Main Program
print("=====")
print("=== Simple Banking ===")
print("=====")

# Create one account
my_account = BankAccount("101", "Tayyab", 5000.0)
```

```

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        amt = float(input("Enter deposit amount: "))
        if amt > 0:
            my_account.deposit(amt)
            print_receipt("Deposit", amt, my_account.balance)
        else:
            print("Amount must be positive.")

    elif choice == "2":
        amt = float(input("Enter withdrawal amount: "))
        if amt > 0:
            success = my_account.withdraw(amt)
            if success:
                print_receipt("Withdrawal", amt, my_account.balance)
            else:
                print("Amount must be positive.")

    elif choice == "3":
        my_account.display_balance()

    elif choice == "4":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```

=====
=== Simple Banking ===
=====

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter choice: 2
Enter withdrawal amount: 1000

=== Withdrawal Receipt ===
Transaction:    Withdrawal
Amount:         $1000.0
=====
New Balance:    $0.0
=====

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter choice: 4

=== Goodbye ===

```

## Program 2: Library Management

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, title, author, isbn):
        new_book = Book(title, author, isbn)
        self.books.append(new_book)
        print("\nBook Added Successfully!")

    def show_all_books(self):
        print("\n=== All Books in Library ===")
        for book in self.books:
            status = "Available"
            if book.available == False:
                status = "Checked Out"
            print(f"{book.title} by {book.author} (ISBN: {book.isbn}) - [{status}]")
        print("=====")

    def borrow_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if book.available:
                    book.available = False
                    return book
                else:
                    print("Book is already borrowed.")
                    return None
        print("Book not found.")
        return None

    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                book.available = True
                return book
        return None

    def print_receipt(action, book_title):
        print(f"\n=== {action} Receipt ===")
        print(f"Action:\t\t{action}")
        print(f"Book:\t\t{book_title}")
        print("=====")
        print("Status:\t\tSuccess")
        print("=====")
```



```

# Main Program
print("=====")
print("=== Library ===")
print("=====")
my_library = Library()

# Add a few dummy books to start
my_library.add_book("Python Basics", "John Doe", "101")
my_library.add_book("Science 101", "Jane Smith", "102")

while True:
    print("\n1. Show All Books")
    print("2. Add Book")
    print("3. Borrow Book")
    print("4. Return Book")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        my_library.show_all_books()

    elif choice == "2":
        t = input("Enter Title: ")
        a = input("Enter Author: ")
        i = input("Enter ISBN: ")
        my_library.add_book(t, a, i)

    elif choice == "3":
        i = input("Enter ISBN to borrow: ")
        book = my_library.borrow_book(i)
        if book:
            print_receipt("Borrow", book.title)

    elif choice == "4":
        i = input("Enter ISBN to return: ")
        book = my_library.return_book(i)
        if book:
            print_receipt("Return", book.title)

    elif choice == "5":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== Library ===
=====

Book Added Successfully!

Book Added Successfully!

1. Show All Books
2. Add Book
3. Borrow Book
4. Return Book
5. Exit
Enter choice: 2
Enter Title: Mein Kamph
Enter Author: Hitler
Enter ISBN: 1974

Book Added Successfully!

1. Show All Books
2. Add Book
3. Borrow Book
4. Return Book
5. Exit
Enter choice: 5

=== Goodbye ===
```

### Program 3: Student Grade System

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def calculate_average(self):
        if len(self.grades) == 0:
            return 0
        total = sum(self.grades)
        return total / len(self.grades)

    def get_letter_grade(self, average):
        if average >= 90:
            return "A"
        elif average >= 80:
            return "B"
        elif average >= 70:
            return "C"
        elif average >= 60:
            return "D"
        else:
            return "F"

    def display_info(self):
        avg = self.calculate_average()
        letter = self.get_letter_grade(avg)

        print("\n=== Student Report Card ===")
        print(f"Name:\t\t{self.name}")
        print(f"ID:\t\t{self.student_id}")
        print("=====")
        print(f"Total Grades:\t{len(self.grades)}")
        print(f"Average:\t{avg}")
        print(f"Letter Grade:\t{letter}")
        print("=====")

def print_receipt(grade):
    print("\n=== Grade Added ===")
    print(f"Score:\t\t{grade}")
    print("=====")
    print("Status:\t\tSaved")
    print("=====")

# Main Program
print("=====")
print("=== Student Grade System ===")
print("=====")
```

```

# Create a student
s_id = input("Enter Student ID: ")
s_name = input("Enter Student Name: ")
student = Student(s_id, s_name)

while True:
    print("\n1. Add Grade")
    print("2. Show Report Card")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        score = float(input("Enter grade (0-100): "))
        if 0 <= score <= 100:
            student.add_grade(score)
            print_receipt(score)
        else:
            print("Grade must be between 0 and 100.")

    elif choice == "2":
        student.display_info()

    elif choice == "3":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```

=====
=== Student Grade System ===
=====
Enter Student ID: 19
Enter Student Name: Tayyab

1. Add Grade
2. Show Report Card
3. Exit
Enter choice: 1
Enter grade (0-100): 95

=== Grade Added ===
Score:          95.0
=====
Status:          Saved
=====

1. Add Grade
2. Show Report Card
3. Exit
Enter choice: 2

=== Student Report Card ===
Name:           Tayyab
ID:              19
=====
Total Grades:    1
Average:         95.0
Letter Grade:    A
=====

1. Add Grade
2. Show Report Card
3. Exit
Enter choice: 3

=== Goodbye ===

```

## Program 4: Restaurant System

```
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category

class Order:
    def __init__(self):
        self.items = []
        self.tax_rate = 0.10 # 10% tax

    def add_item(self, item):
        self.items.append(item)

    def calculate_total(self):
        subtotal = 0
        for item in self.items:
            subtotal += item.price
        return subtotal

    def generate_receipt(self):
        subtotal = self.calculate_total()

        # Apply discount (10% off if subtotal > 50)
        discount = 0
        if subtotal > 50:
            discount = subtotal * 0.10

        after_discount = subtotal - discount
        tax = after_discount * self.tax_rate
        final_total = after_discount + tax

        print("\n=== Customer Receipt ===")
        print("Item\t\tPrice")
        print("-----")
        for item in self.items:
            print(f"{item.name}\t\t${item.price}")
        print("-----")
        print(f"Subtotal:\t${subtotal}")
        print(f"Discount:\t-${discount}")
        print(f"Tax (10%):\t${tax}")
        print("=====")
        print(f"Grand Total:\t${final_total}")
        print("=====")

    def print_added_msg(item_name):
        print(f"\n[Success] '{item_name}' added to order.")
```

```

# Main Program
print("=====")
print("=== Restaurant System ===")
print("=====")

my_order = Order()

# Pre-defined Menu
menu = [
    MenuItem("Burger", 10.0, "Main"),
    MenuItem("Pizza", 15.0, "Main"),
    MenuItem("Coke", 2.0, "Drink"),
    MenuItem("Fries", 5.0, "Side"),
]

while True:
    print("\n1. View Menu")
    print("2. Add Item to Order")
    print("3. Finalize Order (Print Receipt)")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("\n=== Menu ===")
        for i in range(len(menu)):
            item = menu[i]
            print(f"{i + 1}. {item.name} - ${item.price} ({item.category})")
        print("=====")

    elif choice == "2":
        item_num = int(input("Enter Menu Number to add: "))
        if item_num > 0 and item_num <= len(menu):
            selected_item = menu[item_num - 1]
            my_order.add_item(selected_item)
            print_added_msg(selected_item.name)
        else:
            print("Invalid menu number.")

    elif choice == "3":
        if len(my_order.items) > 0:
            my_order.generate_receipt()
            # Clear order after printing
            my_order.items = []
            print("\nOrder Completed. Starting new order...")
        else:
            print("Order is empty!")

    elif choice == "4":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== Restaurant System ===
=====

1. View Menu
2. Add Item to Order
3. Finalize Order (Print Receipt)
4. Exit
Enter choice: 1

=== Menu ===
1. Burger - $10.0 (Main)
2. Pizza - $15.0 (Main)
3. Coke - $2.0 (Drink)
4. Fries - $5.0 (Side)
=====

1. View Menu
2. Add Item to Order
3. Finalize Order (Print Receipt)
4. Exit
Enter choice: 2
Enter Menu Number to add: 2

[Success] 'Pizza' added to order.

1. View Menu
2. Add Item to Order
3. Finalize Order (Print Receipt)
4. Exit
Enter choice: 3

=== Customer Receipt ===
Item          Price
-----
Pizza          $15.0
-----
Subtotal:      $15.0
Discount:      -$0
Tax (10%):     $1.5
=====
Grand Total:   $16.5
=====

Order Completed. Starting new order...

1. View Menu
2. Add Item to Order
3. Finalize Order (Print Receipt)
4. Exit
Enter choice: 4

=== Goodbye ===
```

## Program 5: City Rental System

```
class Vehicle:
    def __init__(self, plate, model, price_per_day):
        self.plate = plate
        self.model = model
        self.price_per_day = price_per_day
        self.available = True
        self.type = "Vehicle"

    def rent_vehicle(self):
        if self.available:
            self.available = False
            return True
        return False

    def return_vehicle(self):
        self.available = True

    def calculate_cost(self, days):
        return self.price_per_day * days

# Derived Classes
class Car(Vehicle):
    def __init__(self, plate, model, price_per_day):
        super().__init__(plate, model, price_per_day)
        self.type = "Car"

class Motorcycle(Vehicle):
    def __init__(self, plate, model, price_per_day):
        super().__init__(plate, model, price_per_day)
        self.type = "Motorcycle"

class Truck(Vehicle):
    def __init__(self, plate, model, price_per_day):
        super().__init__(plate, model, price_per_day)
        self.type = "Truck"

def print_receipt(vehicle, days, total_cost):
    print("\n=== Rental Receipt ===")
    print(f"Vehicle:\t{vehicle.type} ({vehicle.model})")
    print(f"Plate:\t\t{vehicle.plate}")
    print(f"Duration:\t{days} Days")
    print("=====")
    print(f"Total Cost:\t${total_cost}")
    print("=====")

# Main Program
print("=====")
print("=== City Rental System ===")
print("=====")
```



```

# Create a fleet of vehicles
fleet = [
    Car("ABC-123", "Toyota Corolla", 50.0),
    Motorcycle("XYZ-999", "Honda CD70", 20.0),
    Truck("TRK-001", "Ford F-150", 100.0),
]

while True:
    print("\n1. View Fleet")
    print("2. Rent Vehicle")
    print("3. Return Vehicle")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("\n=== Available Vehicles ===")
        for i in range(len(fleet)):
            v = fleet[i]
            status = "Available" if v.available else "Rented"
            print(f"{i + 1}. {v.type}: {v.model} (${v.price_per_day}/day) - [{status}]")
        print("=====")

    elif choice == "2":
        v_index = int(input("Enter Vehicle Number: ")) - 1

        if 0 <= v_index < len(fleet):
            vehicle = fleet[v_index]

            if vehicle.available:
                days = int(input("How many days? "))
                if days > 0:
                    vehicle.rent_vehicle()
                    cost = vehicle.calculate_cost(days)
                    print_receipt(vehicle, days, cost)
                else:
                    print("Days must be positive.")
            else:
                print("Sorry, that vehicle is already rented.")
        else:
            print("Invalid vehicle number.")

    elif choice == "3":
        v_index = int(input("Enter Vehicle Number to return: ")) - 1

        if 0 <= v_index < len(fleet):
            vehicle = fleet[v_index]
            if not vehicle.available:
                vehicle.return_vehicle()
                print(f"\n[Success] {vehicle.model} returned successfully.")
            else:
                print("This vehicle is already in the garage.")
        else:
            print("Invalid vehicle number.")

```

```

elif choice == "4":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```

=====
=== City Rental System ===
=====

1. View Fleet
2. Rent Vehicle
3. Return Vehicle
4. Exit
Enter choice: 1

=== Available Vehicles ===
1. Car: Toyota Corolla ($50.0/day) - [Available]
2. Motorcycle: Honda CD70 ($20.0/day) - [Available]
3. Truck: Ford F-150 ($100.0/day) - [Available]
=====

1. View Fleet
2. Rent Vehicle
3. Return Vehicle
4. Exit
Enter choice: 2
Enter Vehicle Number: 1
How many days? 5

=== Rental Receipt ===
Vehicle:      Car (Toyota Corolla)
Plate:       ABC-123
Duration:    5 Days
=====
Total Cost:   $250.0
=====

1. View Fleet
2. Rent Vehicle
3. Return Vehicle
4. Exit
Enter choice: 4

=== Goodbye ===

```

## Program 6: HR Management System

```
class Employee:
    def __init__(self, emp_id, name, position, monthly_salary, years_worked):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.monthly_salary = monthly_salary
        self.years_worked = years_worked

    def get_annual_salary(self):
        return self.monthly_salary * 12

    def apply_raise(self, percentage):
        raise_amount = self.monthly_salary * (percentage / 100)
        self.monthly_salary += raise_amount
        return raise_amount

    def display_details(self):
        print("\n=== Employee Profile ===")
        print(f"ID:\t\t{self.emp_id}")
        print(f"Name:\t\t{self.name}")
        print(f"Position:\t\t{self.position}")
        print(f"Exp (Years):\t\t{self.years_worked}")
        print("=====")
        print(f"Monthly Salary:\t\t${self.monthly_salary}")
        print("=====")

def print_raise_receipt(name, old_salary, new_salary, amount):
    print("\n=== Salary Update Receipt ===")
    print(f"Employee:\t\t{name}")
    print(f"Old Salary:\t\t${old_salary}")
    print(f"Raise Amount:\t\t+${amount}")
    print("=====")
    print(f"New Salary:\t\t${new_salary}")
    print("=====")

def print_annual_report(name, annual_salary):
    print("\n=== Annual Income Report ===")
    print(f"Employee:\t\t{name}")
    print("-----")
    print(f"Total Annual:\t\t${annual_salary}")
    print("=====")

# Main Program
print("=====")
print("=== HR Management System ===")
print("=====")

# Create a sample employee
emp = Employee("E-101", "Tayyab", "Manager", 5000.0, 3)
```

```

while True:
    print("\n1. View Employee Details")
    print("2. Calculate Annual Salary")
    print("3. Give Raise")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        emp.display_details()

    elif choice == "2":
        annual = emp.get_annual_salary()
        print_annual_report(emp.name, annual)

    elif choice == "3":
        percent = float(input("Enter raise percentage (e.g., 10 for 10%): "))
        if percent > 0:
            old_sal = emp.monthly_salary
            raise_amt = emp.apply_raise(percent)
            print_raise_receipt(emp.name, old_sal, emp.monthly_salary,
raise_amt)
        else:
            print("Percentage must be positive.")

    elif choice == "4":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```

=====
=== HR Management System ===
=====

1. View Employee Details
2. Calculate Annual Salary
3. Give Raise
4. Exit
Enter choice: 1

=== Employee Profile ===
ID:           E-101
Name:         Tayyab
Position:     Manager
Exp (Years):  3
=====
Monthly Salary:$5000.0
=====

1. View Employee Details
2. Calculate Annual Salary
3. Give Raise
4. Exit
Enter choice: 4

=== Goodbye ===

```

## Program 7: Supermarket System

```
class Product:
    def __init__(self, p_id, name, price, stock):
        self.p_id = p_id
        self.name = name
        self.price = price
        self.stock = stock

class ShoppingCart:
    def __init__(self):
        self.cart_items = [] # Stores [product, quantity]

    def add_product(self, product, quantity):
        if quantity <= product.stock:
            # Add to cart list
            self.cart_items.append([product, quantity])
            print(f"\n[Success] Added {quantity} x {product.name} to cart.")
        else:
            print("\n[Error] Not enough stock available.")

    def calculate_total(self):
        total = 0
        for item in self.cart_items:
            product = item[0]
            qty = item[1]
            total += product.price * qty
        return total

    def checkout(self):
        if len(self.cart_items) == 0:
            print("Cart is empty.")
            return

        total = self.calculate_total()

        # Update Inventory
        for item in self.cart_items:
            product = item[0]
            qty = item[1]
            product.stock -= qty # Reduce stock
```

```

# Print Final Receipt
print("\n=== Official Receipt ===")
print("Item\t\tQty\tPrice")
print("-----")
for item in self.cart_items:
    p = item[0]
    q = item[1]
    cost = p.price * q
    print(f"{p.name}\t\t{q}\t\t${cost}")
print("-----")
print(f"Total Amount:\t\t${total}")
print("=====")
print("Payment Status:\tPaid")
print("=====")

# Clear cart after checkout
self.cart_items = []

# Main Program
print("=====")
print("=== Supermarket System ===")
print("=====")

# Create Store Inventory
inventory = [
    Product("P01", "Laptop", 1000.0, 5),
    Product("P02", "Mouse", 20.0, 10),
    Product("P03", "Keyboard", 50.0, 8),
]

my_cart = ShoppingCart()

while True:
    print("\n1. View Products")
    print("2. Add to Cart")
    print("3. View Cart Total")
    print("4. Checkout")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("\n=== Available Products ===")
        for i in range(len(inventory)):
            p = inventory[i]
            print(f"{i + 1}. {p.name} - ${p.price} (Stock: {p.stock})")
        print("=====")

```

```

elif choice == "2":
    p_index = int(input("Enter Product Number: ")) - 1

    if 0 <= p_index < len(inventory):
        selected_product = inventory[p_index]
        qty = int(input(f"Enter Quantity for {selected_product.name}: "))

        if qty > 0:
            my_cart.add_product(selected_product, qty)
        else:
            print("Quantity must be positive.")
    else:
        print("Invalid product number.")

elif choice == "3":
    total = my_cart.calculate_total()
    print(f"\nCurrent Cart Total: ${total}")

elif choice == "4":
    my_cart.checkout()

elif choice == "5":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```

=====
=== Supermarket System ===
=====

1. View Products
2. Add to Cart
3. View Cart Total
4. Checkout
5. Exit
Enter choice: 1

=== Available Products ===
1. Laptop - $1000.0 (Stock: 5)
2. Mouse - $20.0 (Stock: 10)
3. Keyboard - $50.0 (Stock: 8)
=====

1. View Products
2. Add to Cart
3. View Cart Total
4. Checkout
5. Exit
Enter choice: 5

=== Goodbye ===

```

## Program 8: City Hospital System

```
class Patient:
    def __init__(self, p_id, name, age, history):
        self.p_id = p_id
        self.name = name
        self.age = age
        self.history = history

class Appointment:
    def __init__(self, patient, doctor, date):
        self.patient = patient
        self.doctor = doctor
        self.date = date

class Clinic:
    def __init__(self):
        self.appointments = []

    def schedule_appointment(self, patient, doctor, date):
        new_appt = Appointment(patient, doctor, date)
        self.appointments.append(new_appt)
        print("\n[Success] Appointment Scheduled.")
        return new_appt

    def cancel_appointment(self, patient_name):
        for appt in self.appointments:
            if appt.patient.name == patient_name:
                self.appointments.remove(appt)
                print(f"\n[Success] Appointment for {patient_name} cancelled.")
                return True
        print("\n[Error] Appointment not found.")
        return False

    def view_appointments(self):
        print("\n=== Daily Schedule ===")
        if len(self.appointments) == 0:
            print("No appointments today.")
        else:
            for appt in self.appointments:
                print(f"Time/Date:\t{appt.date}")
                print(f"Patient:\t{appt.patient.name} (ID: {appt.patient.p_id})")
                print(f"Doctor:\t\t{appt.doctor}")
                print("-----")
            print("=====")

    def print_ticket(appt):
        print("\n=== Appointment Ticket ===")
        print(f"Patient:\t{appt.patient.name}")
        print(f"Age:\t\t{appt.patient.age}")
        print(f"History:\t{appt.patient.history}")
        print("-----")
        print(f"Doctor:\t\t{appt.doctor}")
        print(f>Date:\t\t{appt.date}")
        print("=====")
```



```

# Main Program
print("=====")
print("=== City Hospital System ===")
print("=====")

my_clinic = Clinic()
patients_list = []

# Pre-load a patient
p1 = Patient("101", "John Smith", 30, "Flu")
patients_list.append(p1)

while True:
    print("\n1. Register New Patient")
    print("2. Schedule Appointment")
    print("3. View All Appointments")
    print("4. Cancel Appointment")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        pid = input("Enter Patient ID: ")
        name = input("Enter Name: ")
        age = input("Enter Age: ")
        hist = input("Medical History: ")

        new_p = Patient(pid, name, age, hist)
        patients_list.append(new_p)
        print(f"\n[Success] Patient {name} registered.")

    elif choice == "2":
        name = input("Enter Patient Name to book: ")

        # Find patient object
        found_patient = None
        for p in patients_list:
            if p.name == name:
                found_patient = p
                break

        if found_patient:
            doc = input("Enter Doctor Name: ")
            date = input("Enter Date/Time: ")

            appt = my_clinic.schedule_appointment(found_patient, doc, date)
            print_ticket(appt)
        else:
            print("Patient not found! Please register them first.")

```

```

elif choice == "3":
    my_clinic.view_appointments()

elif choice == "4":
    name = input("Enter Patient Name to cancel: ")
    my_clinic.cancel_appointment(name)

elif choice == "5":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```

=====
=== City Hospital System ===
=====

1. Register New Patient
2. Schedule Appointment
3. View All Appointments
4. Cancel Appointment
5. Exit
Enter choice: 1
Enter Patient ID: 101
Enter Name: Tayyab
Enter Age: 20
Medical History: No

[Success] Patient Tayyab registered.

1. Register New Patient
2. Schedule Appointment
3. View All Appointments
4. Cancel Appointment
5. Exit
Enter choice: 5

=== Goodbye ===

```

## Program 9: Social Media App

```
class Post:
    def __init__(self, author, content, is_private):
        self.author = author
        self.content = content
        self.is_private = is_private # True = Private, False = Public

class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = []
        self.posts = []

    def add_friend(self, friend_name):
        self.friends.append(friend_name)

    def create_post(self, content, privacy_option):
        # privacy_option: 1 for Public, 2 for Private
        is_private = False
        if privacy_option == "2":
            is_private = True

        new_post = Post(self.username, content, is_private)
        self.posts.append(new_post)
        return new_post

    def show_timeline(self, viewer_name):
        print(f"\n=== Timeline: {self.username} ===")

        # Check if viewer is the owner or a friend
        is_friend = False
        if viewer_name in self.friends or viewer_name == self.username:
            is_friend = True

        if len(self.posts) == 0:
            print("No posts yet.")
        else:
            for post in self.posts:
                # Logic: Show if Public OR (Private AND viewer is friend/owner)
                if not post.is_private or is_friend:
                    privacy_label = "Private" if post.is_private else "Public"
                    print(f"[{privacy_label}] {post.content}")
                    print("-----")
            print("=====")

    def print_post_receipt(post):
        print("\n=== Post Created ===")
        print(f"Author:\t\t{post.author}")
        print(f"Content:\t\t{post.content}")
        type_str = "Private" if post.is_private else "Public"
        print(f"Privacy:\t\t{type_str}")
        print("=====")
```

```

def print_friend_receipt(user, friend):
    print("\n=== Friend Added ===")
    print(f"User:\t\t{user}")
    print(f"New Friend:\t{friend}")
    print("=====")

# Main Program
print("=====")
print("=== Social Media App ===")
print("=====")

# Create two users for testing
user1 = User("Tayyab", "tayyab@gmail.com")
user2 = User("Zain", "zain@gmail.com")

# Set current logged in user
current_user = user1

while True:
    print(f"\nLogged in as: {current_user.username}")
    print("1. Create Post")
    print("2. Add Friend")
    print("3. View My Timeline")
    print("4. Switch User")
    print("5. View Current user")
    print("6. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        text = input("Enter post content: ")
        print("Privacy: (1) Public (2) Private")
        priv = input("Select option: ")

        post = current_user.create_post(text, priv)
        print_post_receipt(post)

    elif choice == "2":
        name = input("Enter friend's username: ")
        current_user.add_friend(name)
        print_friend_receipt(current_user.username, name)

    elif choice == "3":
        current_user.show_timeline(current_user.username)

    elif choice == "4":
        current_user = user2
        print(f"\n[Success] Switched to user {user2.username}")

    elif choice == "5":
        user1.show_timeline(current_user.username)

    elif choice == "6":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== Social Media App ===
=====

Logged in as: Tayyab
1. Create Post
2. Add Friend
3. View My Timeline
4. Switch User
5. View Current user)
6. Exit
Enter choice: 4

[Success] Switched to user Zain

Logged in as: Zain
1. Create Post
2. Add Friend
3. View My Timeline
4. Switch User
5. View Current user)
6. Exit
Enter choice: 6

=== Goodbye ===
```

## Program 10: Hotel Management

```
class Room:
    def __init__(self, room_num, r_type, price):
        self.room_num = room_num
        self.r_type = r_type
        self.price = price
        self.available = True

class Hotel:
    def __init__(self):
        self.rooms = []

    def add_room(self, room):
        self.rooms.append(room)

    def show_rooms(self):
        print("\n=== Room Status ===")
        print("Room\tType\t\tPrice\tStatus")
        print("-----")
        for room in self.rooms:
            status = "Available" if room.available else "Booked"
            # Using tabs for alignment
            print(f"{room.room_num}\t{room.r_type}\t\t${room.price}\t{status}")
        print("=====")

    def find_room(self, room_num):
        for room in self.rooms:
            if room.room_num == room_num:
                return room
        return None

    def print_booking_receipt(room, days, total_cost):
        print("\n=== Booking Receipt ===")
        print(f"Room Number:\t{room.room_num}")
        print(f"Room Type:\t{room.r_type}")
        print(f"Duration:\t{days} Night(s)")
        print(f"Price/Night:\t${room.price}")
        print("=====")
        print(f"Total Cost:\t${total_cost}")
        print("=====")

    def print_checkout_receipt(room):
        print("\n=== Checkout Receipt ===")
        print(f"Room:\t\t{room.room_num}")
        print("Status:\t\tChecked Out")
        print("=====")
        print("Thank you for staying!")
        print("=====")

# Main Program
print("=====")
print("=== Hotel Management ===")
print("=====")
```

```

my_hotel = Hotel()

# Setup some rooms
my_hotel.add_room(Room("101", "Single", 50))
my_hotel.add_room(Room("102", "Double", 80))
my_hotel.add_room(Room("201", "Suite ", 150))

while True:
    print("\n1. View All Rooms")
    print("2. Book a Room")
    print("3. Check Out")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        my_hotel.show_rooms()

    elif choice == "2":
        r_num = input("Enter Room Number to book: ")
        room = my_hotel.find_room(r_num)

        if room:
            if room.available:
                days = int(input("Enter number of nights: "))
                if days > 0:
                    room.available = False
                    cost = room.price * days
                    print_booking_receipt(room, days, cost)
                else:
                    print("Nights must be positive.")
            else:
                print("Sorry, this room is already occupied.")
        else:
            print("Room number not found.")

    elif choice == "3":
        r_num = input("Enter Room Number to check out: ")
        room = my_hotel.find_room(r_num)

        if room:
            if not room.available:
                room.available = True
                print_checkout_receipt(room)
            else:
                print("This room is already empty.")
        else:
            print("Room number not found.")

    elif choice == "4":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== Hotel Management ===
=====

1. View All Rooms
2. Book a Room
3. Check Out
4. Exit
Enter choice: 1

=== Room Status ===
Room      Type      Price    Status
-----
101      Single      $50      Available
102      Double      $80      Available
201      Suite       $150     Available
=====

1. View All Rooms
2. Book a Room
3. Check Out
4. Exit
Enter choice: 2
Enter Room Number to book: 103
Room number not found.

1. View All Rooms
2. Book a Room
3. Check Out
4. Exit
Enter choice: 4

=== Goodbye ===
```



## Program 11: University System

```
class Course:
    def __init__(self, code, name, credits, capacity, time_slot):
        self.code = code
        self.name = name
        self.credits = credits
        self.capacity = capacity
        self.time_slot = time_slot
        self.enrolled = 0

class Student:
    def __init__(self, s_id, name):
        self.s_id = s_id
        self.name = name
        self.my_courses = []
        self.total_credits = 0
        self.max_credits = 15 # Limit per semester

    def check_conflict(self, new_time):
        for course in self.my_courses:
            if course.time_slot == new_time:
                return True # Conflict found
        return False

    def register(self, course):
        # 1. Check Capacity
        if course.enrolled >= course.capacity:
            print("\n[Error] Course is full!")
            return False

        # 2. Check Credit Limit
        if (self.total_credits + course.credits) > self.max_credits:
            print(f"\n[Error] Credit limit exceeded! (Max: {self.max_credits})")
            return False

        # 3. Check Schedule Conflict
        if self.check_conflict(course.time_slot):
            print(f"\n[Error] Time conflict! You have a class at {course.time_slot}.")
            return False

        # If all checks pass:
        self.my_courses.append(course)
        self.total_credits += course.credits
        course.enrolled += 1
        return True

    def show_schedule(self):
        print(f"\n=== Schedule for {self.name} ===")
        if len(self.my_courses) == 0:
            print("No courses registered.")
        else:
            print("Code\tName\t\tTime")
            print("-----")
            for c in self.my_courses:
                print(f"{c.code}\t{c.name}\t{c.time_slot}")
            print("-----")
            print(f"Total Credits: {self.total_credits}/{self.max_credits}")
        print("=====")
```

```

def print_receipt(course):
    print("\n=== Registration Success ===")
    print(f"Course:\t\t{course.code}")
    print(f"Name:\t\t{course.name}")
    print(f"Time:\t\t{course.time_slot}")
    print(f"Credits:\t{course.credits}")
    print("=====")

# Main Program
print("=====")
print("=== University System ===")
print("=====")

# Create Course Catalog
catalog = [
    Course("CS101", "Programming Fundamentals", 3, 30, "Mon 10am"),
    Course("CS102", "OOP", 4, 30, "Mon 10am"), # Conflicts with CS101
    Course("ENG101", "English ", 3, 50, "Tue 2pm"),
    Course("MATH1", "Calculus", 4, 40, "Wed 9am"),
]

# Create a student
student = Student("019", "Tayyab")

while True:
    print("\n1. View Course Catalog")
    print("2. Register for Course")
    print("3. View My Schedule")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("\n=== Course Catalog ===")
        for i in range(len(catalog)):
            c = catalog[i]
            avail = c.capacity - c.enrolled
            print(
                f"{i + 1}. [{c.code}] {c.name} ({c.credits} Cr) - {c.time_slot} - "
                f"Seats: {avail}"
            )
            print("=====")

        elif choice == "2":
            c_index = int(input("Enter Course Number to register: ")) - 1

            if 0 <= c_index < len(catalog):
                course_to_add = catalog[c_index]

                # Try to register
                success = student.register(course_to_add)
                if success:
                    print_receipt(course_to_add)
                else:
                    print("Invalid course number.")

            elif choice == "3":
                student.show_schedule()

            elif choice == "4":
                print("\n=== Goodbye ===")
                break

            else:
                print("Invalid choice.")

```

## Output:

```
=====
=== University System ===
=====

1. View Course Catalog
2. Register for Course
3. View My Schedule
4. Exit
Enter choice: 1

=== Course Catalog ===
1. [CS101] Programming Fundamentals (3 Cr) - Mon 10am - Seats: 30
2. [CS102] OOP (4 Cr) - Mon 10am - Seats: 30
3. [ENG101] English (3 Cr) - Tue 2pm - Seats: 50
4. [MATH1] Calculus (4 Cr) - Wed 9am - Seats: 40
=====

1. View Course Catalog
2. Register for Course
3. View My Schedule
4. Exit
Enter choice: 2
Enter Course Number to register: 2

=== Registration Success ===
Course:      CS102
Name:       OOP
Time:       Mon 10am
Credits:    4
=====

1. View Course Catalog
2. Register for Course
3. View My Schedule
4. Exit
Enter choice: 4

=== Goodbye ===
```

## Program 12: Warehouse Inventory

```
class Product:
    def __init__(self, p_id, name, qty, price, supplier):
        self.p_id = p_id
        self.name = name
        self.qty = qty
        self.price = price
        self.supplier = supplier

class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, p_id, name, qty, price, supplier):
        new_product = Product(p_id, name, qty, price, supplier)
        self.products.append(new_product)
        print(f"\n[Success] Product '{name}' added to inventory.")

    def update_stock(self, p_id, amount):
        # amount can be positive (restock) or negative (sold)
        for p in self.products:
            if p.p_id == p_id:
                new_qty = p.qty + amount
                if new_qty < 0:
                    print("\n[Error] Cannot remove more than available stock!")
                    return None
                else:
                    p.qty = new_qty
                    return p
        print("\n[Error] Product ID not found.")
        return None

    def view_inventory(self):
        print("\n=== Current Inventory ===")
        if len(self.products) == 0:
            print("Inventory is empty.")
        else:
            print("ID\tName\tQty\tPrice")
            print("-----")
            for p in self.products:
                print(f"{p.p_id}\t{p.name}\t{p.qty}\t${p.price}")
            print("=====")

    def check_low_stock(self):
        print("\n=== Low Stock Alerts ===")
        found = False
        for p in self.products:
            if p.qty < 5: # Alert if less than 5 items
                print(f"[ALERT] {p.name} (ID: {p.p_id}) is low! Qty: {p.qty}")
                print(f"Supplier: {p.supplier}")
                print("-----")
                found = True

        if not found:
            print("All stock levels are good.")
        print("=====")
```

```

def print_update_receipt(product, change):
    action = "Restocked" if change > 0 else "Removed"
    print(f"\n=== Stock Update Receipt ===")
    print(f"Product:\t{product.name}")
    print(f"Action:\t\t{action} {abs(change)}")
    print("-----")
    print(f"New Quantity:\t{product.qty}")
    print("=====")

# Main Program
print("=====")
print("=== Warehouse Inventory ===")
print("=====")

my_inventory = Inventory()

# Add dummy items
my_inventory.add_product("101", "Drill", 10, 50.0, "ToolCorp")
my_inventory.add_product("102", "Hammer", 3, 15.0, "BuildIt") # Low stock

while True:
    print("\n1. View Inventory")
    print("2. Add New Product")
    print("3. Update Stock (Add/Remove)")
    print("4. Check Low Stock")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        my_inventory.view_inventory()

    elif choice == "2":
        pid = input("Enter Product ID: ")
        name = input("Enter Name: ")
        qty = int(input("Enter Initial Quantity: "))
        price = float(input("Enter Price: "))
        sup = input("Enter Supplier: ")

        my_inventory.add_product(pid, name, qty, price, sup)

    elif choice == "3":
        pid = input("Enter Product ID to update: ")
        print("Enter positive number to add, negative to remove.")
        amount = int(input("Enter quantity change: "))

        updated_product = my_inventory.update_stock(pid, amount)
        if updated_product:
            print_update_receipt(updated_product, amount)

    elif choice == "4":
        my_inventory.check_low_stock()

    elif choice == "5":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== Warehouse Inventory ===
=====

[Success] Product 'Drill' added to inventory.

[Success] Product 'Hammer' added to inventory.

1. View Inventory
2. Add New Product
3. Update Stock (Add/Remove)
4. Check Low Stock
5. Exit
Enter choice: 1

=== Current Inventory ===
ID      Name      Qty      Price
-----
101     Drill        10     $50.0
102     Hammer         3     $15.0
=====

1. View Inventory
2. Add New Product
3. Update Stock (Add/Remove)
4. Check Low Stock
5. Exit
Enter choice: 4

=== Low Stock Alerts ===
[ALERT] Hammer (ID: 102) is low! Qty: 3
        Supplier: BuildIt
-----
=====

1. View Inventory
2. Add New Product
3. Update Stock (Add/Remove)
4. Check Low Stock
5. Exit
Enter choice: 5

=== Goodbye ===
```

## Program 13: City Cinema System

```
class Movie:
    def __init__(self, title, genre, showtime):
        self.title = title
        self.genre = genre
        self.showtime = showtime
        self.seats = [True] * 10

class Theater:
    def __init__(self):
        self.movies = []

    def add_movie(self, title, genre, time):
        new_movie = Movie(title, genre, time)
        self.movies.append(new_movie)

    def show_movies(self):
        print("\n=== Now Showing ===")
        for i in range(len(self.movies)):
            m = self.movies[i]
            print(f"{i + 1}. {m.title} ({m.genre}) - {m.showtime}")
        print("=====")

    def view_seats(self, movie):
        print(f"\n=== Seating for {movie.title} ===")
        print("S = Standard ($10) | P = Premium ($15) | X = Booked")
        print("-----")

        # Display seats nicely
        line = ""
        for i in range(10):
            seat_num = i + 1
            if movie.seats[i] == False:
                status = "[X]" # Booked
            elif seat_num <= 5:
                status = f"[S{seat_num}]" # Standard
            else:
                status = f"[P{seat_num}]" # Premium
            line += status + " "
        print(line)
        print("=====")

    def book_seat(self, movie, seat_num):
        index = seat_num - 1

        # Check if valid seat number
        if index < 0 or index >= 10:
            print("Invalid seat number.")
            return 0

        # Check if already booked
        if movie.seats[index] == False:
            print("Sorry, that seat is already taken.")
            return 0
```

```

        # Calculate Price
        price = 0
        category = ""
        if seat_num <= 5:
            price = 10
            category = "Standard"
        else:
            price = 15
            category = "Premium"

        # Book the seat
        movie.seats[index] = False
        return price, category

def print_ticket(movie, seat_num, category, price):
    print("\n=== Cinema Ticket ===")
    print(f"Movie:\t\t{movie.title}")
    print(f"Time:\t\t{movie.showtime}")
    print(f"Seat:\t\t#{seat_num} ({category})")
    print("-----")
    print(f"Price:\t\t${price}")
    print("=====")
    print("Enjoy the show!")
    print("=====")

# Main Program
print("=====")
print("=== City Cinema System ===")
print("=====")

cinema = Theater()

# Add movies
cinema.add_movie("Avengers", "Action", "5:00 PM")
cinema.add_movie("Frozen 2", "Animation", "7:00 PM")

while True:
    print("\n1. View Movies")
    print("2. Book a Ticket")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        cinema.show_movies()

    elif choice == "2":
        # 1. Select Movie
        cinema.show_movies()
        m_index = int(input("Select Movie Number: ")) - 1

        if 0 <= m_index < len(cinema.movies):
            selected_movie = cinema.movies[m_index]

            # 2. Show Seats
            cinema.view_seats(selected_movie)

            # 3. Select Seat
            s_num = int(input("Enter Seat Number to book: "))

            # 4. Process Booking
            result = cinema.book_seat(selected_movie, s_num)

```



```

        if result != 0:
            price = result[0]
            cat = result[1]
            print_ticket(selected_movie, s_num, cat, price)
        else:
            print("Invalid movie selection.")

    elif choice == "3":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```

=====
=== City Cinema System ===
=====

1. View Movies
2. Book a Ticket
3. Exit
Enter choice: 1

=== Now Showing ===
1. Avengers (Action) - 5:00 PM
2. Frozen 2 (Animation) - 7:00 PM
=====

1. View Movies
2. Book a Ticket
3. Exit
Enter choice: 3

=== Goodbye ===

```

## Program 14: City Gym System

```
class Member:
    def __init__(self, m_id, name, m_type):
        self.m_id = m_id
        self.name = name
        self.m_type = m_type # Basic or Premium
        self.attendance = 0

class Trainer:
    def __init__(self, name, specialty):
        self.name = name
        self.specialty = specialty

class Gym:
    def __init__(self):
        self.sessions = []

    def schedule_session(self, member, trainer, time):
        if member.m_type == "Basic":
            print("\n[Error] Only Premium members can book trainers.")
            return False

        # Create a simple record
        session_info = [member, trainer, time]
        self.sessions.append(session_info)
        return True

    def mark_attendance(self, member):
        member.attendance += 1
        print(f"\n[Success] Welcome back, {member.name}!")

    def print_session_receipt(member, trainer, time):
        print("\n=== Session Booked ===")
        print(f"Member:\t\t{member.name}")
        print(f"Trainer:\t\t{trainer.name}")
        print(f"Specialty:\t\t{trainer.specialty}")
        print("-----")
        print(f"Time:\t\t{time}")
        print("=====")

    def print_attendance_report(member):
        print("\n=== Member Stats ===")
        print(f"Name:\t\t{member.name}")
        print(f"Type:\t\t{member.m_type}")
        print("-----")
        print(f"Total Visits:\t\t{member.attendance}")
        print("=====")

# Main Program
print("=====")
print("=== City Gym System ===")
print("=====")

gym = Gym()
```

```

# Create dummy data
members = [Member("101", "Tayyab", "Basic"), Member("102", "Haseeb", "Premium")]

trainers = [Trainer("Coach Zain", "Weightlifting"), Trainer("Coach Muneeb", "Yoga")]

while True:
    print("\n1. View Members")
    print("2. View Trainers")
    print("3. Schedule Session")
    print("4. Mark Attendance")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("\n=== Member List ===")
        for i in range(len(members)):
            m = members[i]
            print(f"{i + 1}. {m.name} ({m.m_type}) - ID: {m.m_id}")
        print("=====")

    elif choice == "2":
        print("\n=== Trainer List ===")
        for i in range(len(trainers)):
            t = trainers[i]
            print(f"{i + 1}. {t.name} - {t.specialty}")
        print("=====")

    elif choice == "3":
        # Select Member
        print("Select Member:")
        for i in range(len(members)):
            print(f"{i + 1}. {members[i].name}")
        m_idx = int(input("Enter number: ")) - 1

        # Select Trainer
        print("Select Trainer:")
        for i in range(len(trainers)):
            print(f"{i + 1}. {trainers[i].name}")
        t_idx = int(input("Enter number: ")) - 1

        if 0 <= m_idx < len(members) and 0 <= t_idx < len(trainers):
            mem = members[m_idx]
            trn = trainers[t_idx]
            time = input("Enter Time (e.g., 5pm): ")

            if gym.schedule_session(mem, trn, time):
                print_session_receipt(mem, trn, time)
        else:
            print("Invalid selection.")

    elif choice == "4":
        m_id = input("Enter Member ID to mark attendance: ")
        found = False
        for m in members:
            if m.m_id == m_id:
                gym.mark_attendance(m)
                print_attendance_report(m)
                found = True
                break
        if not found:
            print("Member ID not found.")

    elif choice == "5":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== City Gym System ===
=====

1. View Members
2. View Trainers
3. Schedule Session
4. Mark Attendance
5. Exit
Enter choice: 1

=== Member List ===
1. Tayyab (Basic) - ID: 101
2. Haseeb (Premium) - ID: 102
=====

1. View Members
2. View Trainers
3. Schedule Session
4. Mark Attendance
5. Exit
Enter choice: 2

=== Trainer List ===
1. Coach Zain - Weightlifting
2. Coach Muneeb - Yoga
=====

1. View Members
2. View Trainers
3. Schedule Session
4. Mark Attendance
5. Exit
Enter choice: 5

=== Goodbye ===
```

## Program 15: Advance Banking System

```
class BankAccount:
    def __init__(self, acc_num, name, balance):
        self.acc_num = acc_num
        self.name = name
        self.balance = balance
        self.type = "Generic"

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            return True
        return False

    def display_info(self):
        print(f"\n=== {self.type} Account ===")
        print(f"Number:\t\t{self.acc_num}")
        print(f"Holder:\t\t{self.name}")
        print("-----")
        print(f"Balance:\t${self.balance}")
        print("=====")

class SavingsAccount(BankAccount):
    def __init__(self, acc_num, name, balance):
        super().__init__(acc_num, name, balance)
        self.type = "Savings"
        self.interest_rate = 0.05 # 5% interest
        self.min_balance = 50.0 # Cannot go below $50

    def withdraw(self, amount):
        if (self.balance - amount) < self.min_balance:
            print("\n[Error] Cannot withdraw. Must maintain $50 minimum.")
            return False
        return super().withdraw(amount)

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.deposit(interest)
        return interest

class CheckingAccount(BankAccount):
    def __init__(self, acc_num, name, balance):
        super().__init__(acc_num, name, balance)
        self.type = "Checking"
        self.fee = 1.0 # $1 fee per withdrawal

    def withdraw(self, amount):
        total_deduction = amount + self.fee
        if total_deduction > self.balance:
            print("\n[Error] Insufficient funds (Amount + $1 Fee).")
            return False

        self.balance -= total_deduction
        print(f"[Note] A transaction fee of ${self.fee} was applied.")
        return True
```

```

def transfer_funds(sender, receiver, amount):
    print("\n... Processing Transfer ...")
    if sender.withdraw(amount):
        receiver.deposit(amount)
        return True
    return False

def print_transfer_receipt(sender, receiver, amount):
    print("\n=== Transfer Receipt ===")
    print(f"From:\t\t{sender.name} ({sender.type})")
    print(f"To:\t\t{receiver.name} ({receiver.type})")
    print(f"Amount:\t\t${amount}")
    print("-----")
    print("Status:\t\tSuccess")
    print("=====")

def print_interest_receipt(acc, amount):
    print("\n=== Interest Earned ===")
    print(f"Account:\t{acc.acc_num}")
    print(f"Rate:\t\t5%")
    print(f"Earned:\t\t+${amount}")
    print("-----")
    print(f"New Balance:\t${acc.balance}")
    print("=====")

# Main Program
print("=====")
print("=== Advanced Banking System ===")
print("=====")

# Create two different accounts
acc1 = SavingsAccount("19", "Tayyab", 500.0)
acc2 = CheckingAccount("33", "Haseeb", 1000.0)

while True:
    print("\n--- Main Menu ---")
    print(f"1. View {acc1.name}'s Account (Savings)")
    print(f"2. View {acc2.name}'s Account (Checking)")
    print("3. Transfer Funds")
    print("4. Transfer Funds")
    print("5. Apply Interest")
    print("6. Withdraw from Checking")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        acc1.display_info()

    elif choice == "2":
        acc2.display_info()

    elif choice == "3":
        amt = float(input("Enter amount to transfer to Sara: "))
        if transfer_funds(acc1, acc2, amt):
            print_transfer_receipt(acc1, acc2, amt)

    elif choice == "4":
        amt = float(input("Enter amount to transfer to Ali: "))
        if transfer_funds(acc2, acc1, amt):
            print_transfer_receipt(acc2, acc1, amt)

```

```

elif choice == "5":
    # Only Savings accounts get interest
    earned = acc1.add_interest()
    print_interest_receipt(acc1, earned)

elif choice == "6":
    amt = float(input("Enter withdrawal amount: "))
    if acc2.withdraw(amt):
        print(f"\n[Success] New Balance: ${acc2.balance}")

elif choice == "7":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```

=====
=== Advanced Banking System ===
=====

--- Main Menu ---
1. View Tayyab's Account (Savings)
2. View Haseeb's Account (Checking)
3. Transfer Funds
4. Transfer Funds
5. Apply Interest
6. Withdraw from Checking
7. Exit
Enter choice: 1

=== Savings Account ===
Number:      19
Holder:      Tayyab
-----
Balance:     $500.0
=====

--- Main Menu ---
1. View Tayyab's Account (Savings)
2. View Haseeb's Account (Checking)
3. Transfer Funds
4. Transfer Funds
5. Apply Interest
6. Withdraw from Checking
7. Exit
Enter choice: 7

=== Goodbye ===

```

## Program 16: Pizza Order System

```
class Pizza:
    def __init__(self, size, crust):
        self.size = size
        self.crust = crust
        self.toppings = []

    def add_topping(self, topping):
        self.toppings.append(topping)

    def calculate_price(self):
        # Base prices
        price = 0
        if self.size == "Small":
            price = 8.0
        elif self.size == "Medium":
            price = 10.0
        elif self.size == "Large":
            price = 12.0

        # Toppings cost $1.50 each
        topping_cost = len(self.toppings) * 1.50

        return price + topping_cost

def print_receipt(pizza, total_price):
    print("\n=== Pizza Receipt ===")
    print(f"Size:\t\t{pizza.size}")
    print(f"Crust:\t\t{pizza.crust}")
    print("Toppings:")

    if len(pizza.toppings) == 0:
        print(" - (Cheese only)")
    else:
        for t in pizza.toppings:
            print(f" - {t}")

    print("-----")
    print(f"Total Price:\t${total_price}")
    print("=====")

# Main Program
print("=====")
print("=== Pizza Order System ===")
print("=====")

while True:
    print("\n1. Create Custom Pizza")
    print("2. Exit")

    choice = input("Enter choice: ")
```



```

if choice == "1":
    # 1. Select Size
    print("\n--- Select Size ---")
    print("1. Small ($8)")
    print("2. Medium ($10)")
    print("3. Large ($12)")
    s_choice = input("Choose size (1-3): ")

    size = "Medium" # Default
    if s_choice == "1":
        size = "Small"
    elif s_choice == "3":
        size = "Large"

    # 2. Select Crust
    print("\n--- Select Crust ---")
    print("1. Thin Crust")
    print("2. Thick Crust")
    c_choice = input("Choose crust (1-2): ")

    crust = "Thick" # Default
    if c_choice == "1":
        crust = "Thin"

    # Create the object
    my_pizza = Pizza(size, crust)

    # 3. Add Toppings
    print("\n--- Add Toppings ($1.50 each) ---")
    print("Type 'done' to finish adding toppings.")

    while True:
        t = input("Enter topping (e.g., Pepperoni, Mushroom): ")
        if t.lower() == "done":
            break
        my_pizza.add_topping(t)
        print(f"Added {t}.")

    # 4. Checkout
    total = my_pizza.calculate_price()
    print_receipt(my_pizza, total)

elif choice == "2":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```
=====
=== Pizza Order System ===
=====

1. Create Custom Pizza
2. Exit
Enter choice: 1

--- Select Size ---
1. Small ($8)
2. Medium ($10)
3. Large ($12)
Choose size (1-3): 2

--- Select Crust ---
1. Thin Crust
2. Thick Crust
Choose crust (1-2): 2

--- Add Toppings ($1.50 each) ---
Type 'done' to finish adding toppings.
Enter topping (e.g., Pepperoni, Mushroom): Mushroom
Added Mushroom.
Enter topping (e.g., Pepperoni, Mushroom): done

=== Pizza Receipt ===
Size:           Medium
Crust:           Thick
Toppings:
  - Mushroom
-----
Total Price:    $11.5
=====

1. Create Custom Pizza
2. Exit
Enter choice: 2

=== Goodbye ===
```

## Program 17: Auto Factory System

```
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price
        self.features = []
        self.qc_passed = False # Quality Control Status

    def add_feature(self, feature):
        self.features.append(feature)

class CarFactory:
    def __init__(self, name):
        self.name = name
        self.production_line = []

    def manufacture_car(self, model, year, color, price):
        new_car = Car(model, year, color, price)
        self.production_line.append(new_car)
        return new_car

    def run_quality_control(self, car_index):
        if 0 <= car_index < len(self.production_line):
            car = self.production_line[car_index]

            if car.qc_passed:
                print("\n[Info] This car has already passed QC.")
                return None

            # For simplicity, we assume QC always passes if run
            car.qc_passed = True
            return car
        else:
            print("\n[Error] Invalid Car ID.")
            return None

    def show_production_log(self):
        print(f"\n=== {self.name} Production Log ===")
        if len(self.production_line) == 0:
            print("No cars produced yet.")
        else:
            print("ID\tModel\t\t\tStatus\t\t\tFeatures")
            print("-----")
            for i in range(len(self.production_line)):
                c = self.production_line[i]
                status = "READY" if c.qc_passed else "PENDING QC"
                # Join features list into a string
                feat_str = ", ".join(c.features) if c.features else "Standard"
                print(f"{i + 1}\t{c.model}\t\t\t{status}\t\t\t{feat_str}")
            print("=====")
```

```

def print_receipt(car):
    print("\n=== Production Receipt ===")
    print(f"Model:\t\t{car.model}")
    print(f"Year:\t\t{car.year}")
    print(f"Color:\t\t{car.color}")
    print(f"Base Price:\t${car.price}")
    print("-----")
    print("Features Added:")
    if car.features:
        for f in car.features:
            print(f"  + {f}")
    else:
        print("  (None)")
    print("=====")

def print_qc_report(car):
    print("\n=== QC Inspection Report ===")
    print(f"Vehicle:\t{car.model} ({car.year})")
    print(f"Inspection:\tPASSED")
    print("-----")
    print("Status:\t\tReady for Sale")
    print("=====")

# Main Program
print("=====")
print("=== Auto Factory System ===")
print("=====")

factory = CarFactory("Tesla Factory")

while True:
    print("\n1. Produce New Car")
    print("2. Run Quality Control (QC)")
    print("3. View Production Log")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        m = input("Enter Model Name: ")
        y = input("Enter Year: ")
        c = input("Enter Color: ")
        p = float(input("Enter Price: "))

        # Create the car
        new_car = factory.manufacture_car(m, y, c, p)

        # Add features loop
        print("Add features (type 'done' to finish):")
        while True:
            f = input("Feature > ")
            if f.lower() == "done":
                break
            new_car.add_feature(f)

        print_receipt(new_car)

    elif choice == "2":
        factory.show_production_log()
        c_id = int(input("Enter Car ID to inspect: ")) - 1

        checked_car = factory.run_quality_control(c_id)
        if checked_car:
            print_qc_report(checked_car)

```

```

elif choice == "3":
    factory.show_production_log()

elif choice == "4":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```

=====
=== Auto Factory System ===
=====

1. Produce New Car
2. Run Quality Control (QC)
3. View Production Log
4. Exit
Enter choice: 1
Enter Model Name: Mehran
Enter Year: 2025
Enter Color: White
Enter Price: 100000
Add features (type 'done' to finish):
Feature > done

=== Production Receipt ===
Model:      Mehran
Year:       2025
Color:      White
Base Price: $100000.0
-----
Features Added:
  (None)
=====

1. Produce New Car
2. Run Quality Control (QC)
3. View Produc4ion Log
4. Exit
=== Goodbye ===

```

## Program 18: School System

```
class Student:
    def __init__(self, s_id, name):
        self.s_id = s_id
        self.name = name
        self.attendance = 0
        self.grades = []

    def get_average(self):
        if len(self.grades) == 0:
            return 0
        total = 0
        for item in self.grades:
            score = item[1]
            total += score
        return total / len(self.grades)

class Classroom:
    def __init__(self, teacher_name, subject):
        self.teacher_name = teacher_name
        self.subject = subject
        self.students = []

    def add_student(self, s_id, name):
        new_student = Student(s_id, name)
        self.students.append(new_student)
        print(f"\n[Success] Student {name} enrolled.")

    def mark_attendance(self, s_id):
        for s in self.students:
            if s.s_id == s_id:
                s.attendance += 1
            return s
        return None

    def add_grade(self, s_id, assignment, score):
        for s in self.students:
            if s.s_id == s_id:
                s.grades.append([assignment, score])
            return s
        return None

    def generate_report(self):
        print(f"\n=== Class Report: {self.subject} ===")
        print(f"Teacher:\t{self.teacher_name}")
        print("-----")
        print("ID\tName\t\tAttendance\tAverage")
        print("-----")

        if len(self.students) == 0:
            print("(No students enrolled)")
        else:
            for s in self.students:
                avg = s.get_average()
                print(f"{s.s_id}\t\t{s.name}\t\t{s.attendance} Days\t\t{avg:.1f}%")
        print("=====")
```

```

def print_grade_receipt(student, assignment, score):
    print("\n=== Grade Recorded ===")
    print(f"Student:\t{student.name}")
    print(f"Task:\t\t{assignment}")
    print(f"Score:\t\t{score}")
    print("=====")

def print_attendance_receipt(student):
    print("\n=== Attendance Updated ===")
    print(f"Student:\t{student.name}")
    print(f"Total Days:\t{student.attendance}")
    print("=====")

# Main Program
print("=====")
print("=== School System 1.0 ===")
print("=====")

# Setup Class
my_class = Classroom("Mr. Samraiz", "PFAI")

# Add some dummy students
my_class.add_student("01", "Tayyab")
my_class.add_student("02", "Haseeb")

while True:
    print(f"\n--- Class: {my_class.subject} ---")
    print("1. View Class Report")
    print("2. Add New Student")
    print("3. Mark Attendance")
    print("4. Add Grade")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        my_class.generate_report()

    elif choice == "2":
        sid = input("Enter Student ID: ")
        name = input("Enter Name: ")
        my_class.add_student(sid, name)

    elif choice == "3":
        sid = input("Enter Student ID to mark present: ")
        student = my_class.mark_attendance(sid)

        if student:
            print_attendance_receipt(student)
        else:
            print("Student ID not found.")

    elif choice == "4":
        sid = input("Enter Student ID: ")
        # Check if student exists before asking for details
        found = False
        for s in my_class.students:
            if s.s_id == sid:
                found = True
                break

        if found:
            assign = input("Assignment Name (e.g., Quiz 1): ")
            score = float(input("Enter Score (0-100): "))

```

```

        updated_student = my_class.add_grade(sid, assign, score)
        if updated_student:
            print_grade_receipt(updated_student, assign, score)
    else:
        print("Student ID not found.")

elif choice == "5":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```

=====
=== School System 1.0 ===
=====

[Success] Student Tayyab enrolled.

[Success] Student Haseeb enrolled.

--- Class: PFAI ---
1. View Class Report
2. Add New Student
3. Mark Attendance
4. Add Grade
5. Exit
Enter choice: 1

=== Class Report: PFAI ===
Teacher:      Mr. Samraiz
-----
ID      Name      Attendance      Average
-----
01      Tayyab      0 Days  0.0%
02      Haseeb      0 Days  0.0%
=====

--- Class: PFAI ---
1. View Class Report
2. Add New Student
3. Mark Attendance
4. Add Grade
5. Exit
Enter choice: 5

=== Goodbye ===

```



## Program 19: Airline System

```
class Passenger:
    def __init__(self, name, passport):
        self.name = name
        self.passport = passport

class Flight:
    def __init__(self, flight_num, destination, time, capacity):
        self.flight_num = flight_num
        self.destination = destination
        self.time = time
        self.capacity = capacity
        self.passengers = []

    def book_seat(self, passenger):
        if len(self.passengers) < self.capacity:
            self.passengers.append(passenger)
            return True
        else:
            return False

    def show_manifest(self):
        print(f"\n=== Manifest: {self.flight_num} ===")
        print(f"Destination:\t{self.destination}")
        print(f"Occupancy:\t{len(self.passengers)}/{self.capacity}")
        print("-----")
        if len(self.passengers) == 0:
            print("No passengers booked.")
        else:
            for i in range(len(self.passengers)):
                p = self.passengers[i]
                print(f"{i + 1}. {p.name} (Passport: {p.passport})")
            print("=====")

def print_ticket(flight, passenger):
    print("\n=== Boarding Pass ===")
    print(f"Passenger:\t{passenger.name}")
    print(f"Flight:\t\t{flight.flight_num}")
    print(f"To:\t\t{flight.destination}")
    print(f"Time:\t\t{flight.time}")
    print("-----")
    print("Status:\t\tConfirmed")
    print("=====")

# Main Program
print("=====")
print("=== Airline System 1.0 ===")
print("=====")

# Create a list of flights
flights = [
    Flight("PK-303", "Dubai", "10:00 AM", 5), # Small capacity for testing
    Flight("PK-786", "London", "02:00 PM", 200),
]
```

```

while True:
    print("\n1. View Flights")
    print("2. Book a Ticket")
    print("3. View Flight Manifest")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("\n=== Available Flights ===")
        for i in range(len(flights)):
            f = flights[i]
            seats_left = f.capacity - len(f.passengers)
            print(
                f"{i + 1}. {f.flight_num} to {f.destination} ({f.time}) - Seats: {seats_left}"
            )
        print("=====")

    elif choice == "2":
        f_index = int(input("Enter Flight Number (from list): ")) - 1

        if 0 <= f_index < len(flights):
            selected_flight = flights[f_index]

            # Check availability before asking for details
            if len(selected_flight.passengers) < selected_flight.capacity:
                name = input("Enter Passenger Name: ")
                ppt = input("Enter Passport Number: ")

                new_passenger = Passenger(name, ppt)

                if selected_flight.book_seat(new_passenger):
                    print_ticket(selected_flight, new_passenger)
                else:
                    print("\n[Error] Flight is full!")
            else:
                print("Invalid flight selection.")

    elif choice == "3":
        f_index = int(input("Enter Flight Number to view: ")) - 1

        if 0 <= f_index < len(flights):
            flights[f_index].show_manifest()
        else:
            print("Invalid flight selection.")

    elif choice == "4":
        print("\n=== Goodbye ===")
        break

    else:
        print("Invalid choice.")

```

## Output:

```
=====
=== Airline System 1.0 ===
=====

1. View Flights
2. Book a Ticket
3. View Flight Manifest
4. Exit
Enter choice: 1

=== Available Flights ===
1. PK-303 to Dubai (10:00 AM) - Seats: 5
2. PK-786 to London (02:00 PM) - Seats: 200
=====

1. View Flights
2. Book a Ticket
3. View Flight Manifest
4. Exit
Enter choice: 3
Enter Flight Number to view: 1

=== Manifest: PK-303 ===
Destination:   Dubai
Occupancy:     0/5
-----
No passengers booked.
=====

1. View Flights
2. Book a Ticket
3. View Flight Manifest
4. Exit
Enter choice: 4

=== Goodbye ===
```

## Program 20: Smart Home System

```
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "OFF" # Default status

    def turn_on(self):
        self.status = "ON"

    def turn_off(self):
        self.status = "OFF"

# Derived Classes
class SmartLight(SmartDevice):
    def __init__(self, name):
        super().__init__(name)
        self.type = "Light"
        self.brightness = 50

class Thermostat(SmartDevice):
    def __init__(self, name):
        super().__init__(name)
        self.type = "Thermostat"
        self.temperature = 22 # Celsius

class SecurityCamera(SmartDevice):
    def __init__(self, name):
        super().__init__(name)
        self.type = "Camera"
        self.recording = False

class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)

    def show_status(self):
        print("\n=== Home Dashboard ===")
        print("ID\tDevice\t\tType\t\tStatus\tDetail")
        print("-----")
        for i in range(len(self.devices)):
            d = self.devices[i]
            detail = ""
            if d.type == "Light":
                detail = f"Bright: {d.brightness}%"
            elif d.type == "Thermostat":
                detail = f"Temp: {d.temperature}C"
            elif d.type == "Camera":
                rec = "REC" if d.recording else "Idle"
                detail = f"Mode: {rec}"

            print(f"{i + 1}\t{d.name}\t{d.type}\t{d.status}\t{detail}")
        print("=====")
```

```

def run_routine(self, routine_name):
    print(f"\n... Executing Routine: {routine_name} ...")

    if routine_name == "Goodnight":
        # Turn off lights, Turn on cameras
        for d in self.devices:
            if d.type == "Light":
                d.turn_off()
            elif d.type == "Camera":
                d.turn_on()
                d.recording = True

    elif routine_name == "Morning":
        # Turn on lights, Turn off cameras
        for d in self.devices:
            if d.type == "Light":
                d.turn_on()
                d.brightness = 100
            elif d.type == "Camera":
                d.turn_off()
                d.recording = False

def print_action_receipt(device, action):
    print("\n=== Action Complete ===")
    print(f"Device:\t\t{device.name}")
    print(f"Action:\t\t{action}")
    print("-----")
    print(f"New Status:\t{device.status}")
    print("=====")

# Main Program
print("=====")
print("=== Smart Home Hub 1.0 ===")
print("=====")

home = SmartHome()

# Add initial devices
home.add_device(SmartLight("Living Room"))
home.add_device(Thermostat("Hallway AC"))
home.add_device(SecurityCamera("Front Door"))

while True:
    print("\n1. View Dashboard")
    print("2. Control a Device")
    print("3. Run Automation Routine")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        home.show_status()

    elif choice == "2":
        home.show_status()
        idx = int(input("Enter Device ID: ")) - 1

```

```

if 0 <= idx < len(home.devices):
    device = home.devices[idx]

    print(f"\nControlling: {device.name}")
    print("1. Turn ON")
    print("2. Turn OFF")

    # Special options for Thermostat/Light
    if device.type == "Thermostat":
        print("3. Set Temperature")
    elif device.type == "Light":
        print("3. Set Brightness")

    action = input("Select Action: ")

    if action == "1":
        device.turn_on()
        print_action_receipt(device, "Power ON")
    elif action == "2":
        device.turn_off()
        print_action_receipt(device, "Power OFF")
    elif action == "3":
        if device.type == "Thermostat":
            t = int(input("Enter Temperature: "))
            device.temperature = t
            print_action_receipt(device, f"Set Temp to {t}C")
        elif device.type == "Light":
            b = int(input("Enter Brightness (0-100): "))
            device.brightness = b
            print_action_receipt(device, f"Set Bright to {b}%")
    else:
        print("Invalid Device ID.")

elif choice == "3":
    print("\n1. Goodnight Mode (Lights OFF, Cameras ON)")
    print("2. Morning Mode (Lights ON, Cameras OFF)")
    r_choice = input("Select Routine: ")

    if r_choice == "1":
        home.run_routine("Goodnight")
        print("\n[Success] Goodnight routine finished.")
    elif r_choice == "2":
        home.run_routine("Morning")
        print("\n[Success] Morning routine finished.")
    else:
        print("Invalid routine.")

elif choice == "4":
    print("\n=== Goodbye ===")
    break

else:
    print("Invalid choice.")

```

## Output:

```
=====
=== Smart Home Hub 1.0 ===
=====

1. View Dashboard
2. Control a Device
3. Run Automation Routine
4. Exit
Enter choice: 1

=== Home Dashboard ===
ID      Device      Type      Status  Detail
-----
1       Living Room    Light     OFF     Bright: 50%
2       Hallway AC       Thermostat OFF     Temp: 22C
3       Front Door       Camera    OFF     Mode: Idle
=====

1. View Dashboard
2. Control a Device
3. Run Automation Routine
4. Exit
Enter choice: 2

=== Home Dashboard ===
ID      Device      Type      Status  Detail
-----
1       Living Room    Light     OFF     Bright: 50%
2       Hallway AC       Thermostat OFF     Temp: 22C
3       Front Door       Camera    OFF     Mode: Idle
=====

Enter Device ID: 2

Controlling: Hallway AC
1. Turn ON
2. Turn OFF
3. Set Temperature
Select Action: 1

=== Action Complete ===
Device:      Hallway AC
Action:      Power ON
-----
New Status:  ON
=====

1. View Dashboard
2. Control a Device
3. Run Automation Routine
4. Exit
Enter choice: 4

=== Goodbye ===
```

---

## Lab 3

### “NumPy”

#### Program 1: Weather Station

```
import numpy as np

temps_list = [21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

temps_array = np.array(temps_list)

avg_temp = np.mean(temps_array)
max_temp = np.max(temps_array)
min_temp = np.min(temps_array)
temp_range = max_temp - min_temp

print("\n=== Weather Station Report ===")
print("-----")
print(f"Average Temp:\t{avg_temp:.2f} °C")
print(f"Max Temp:\t{max_temp} °C")
print(f"Min Temp:\t{min_temp} °C")
print("-----")
print(f"Temp Range:\t{temp_range:.2f} °C")
print("=====")
```

#### Output:

|                                |          |
|--------------------------------|----------|
| === Weather Station Report === |          |
| -----                          |          |
| Average Temp:                  | 23.13 °C |
| Max Temp:                      | 26.5 °C  |
| Min Temp:                      | 19.8 °C  |
| -----                          |          |
| Temp Range:                    | 6.70 °C  |
| =====                          |          |



## Program 2: Fruit Store Sale Record

```
import numpy as np

data = [
    [12, 15, 14, 10, 9, 8, 7],
    [11, 12, 13, 9, 5, 8, 6],
    [7, 9, 12, 10, 11, 13, 12],
    [10, 11, 12, 7, 8, 9, 11],
]

sales_array = np.array(data)

weekly_totals = np.sum(sales_array, axis=1)
flat_max_index = np.argmax(sales_array)
max_week_idx, max_day_idx = np.unravel_index(flat_max_index, sales_array.shape)
best_week_idx = np.argmax(weekly_totals)

print("\n=== Fruit Store Sales Report ===")
print("Week\t\tTotal Sales")
print("-----")
for i in range(len(weekly_totals)):
    print(f"Week {i + 1}:\t\t{weekly_totals[i]}")
print("=====")
print(f"Highest Sale:\tWeek {max_week_idx + 1}, Day {max_day_idx + 1}")
print(f"Sale Amount:\t\t{sales_array[max_week_idx, max_day_idx]}")
print("-----")
print(f"Best Week:\t\tWeek {best_week_idx + 1}")
print(f"Total Sold:\t\t\t{weekly_totals[best_week_idx]}")
print("=====")
```

### Output:

```
=== Fruit Store Sales Report ===
Week          Total Sales
-----
Week 1:       75
Week 2:       64
Week 3:       74
Week 4:       68
=====
Highest Sale:  Week 1, Day 2
Sale Amount:   15
-----
Best Week:           Week 1
Total Sold:         75
=====
```

### Program 3: Class Performance Report

```
import numpy as np

marks_list = [56, 78, 45, 90, 88, 76, 59, 62]

marks_array = np.array(marks_list)

final_marks = marks_array + 5

pass_count = np.sum(final_marks > 60)

median_score = np.median(final_marks)
std_dev = np.std(final_marks)

print("\n=== Class Performance Report ===")
print(f"Total Students:\t{len(marks_array)}")
print("-----")
print(f"Original:\t{marks_array}")
print(f"With Bonus:\t{final_marks}")
print("-----")
print(f"Students > 60:\t{pass_count}")
print(f"Median Score:\t{median_score}")
print(f"Std Deviation:\t{std_dev:.2f}")
print("=====")
```

#### Output:

```
=== Class Performance Report ===
Total Students:8
-----
Original:      [56 78 45 90 88 76 59 62]
With Bonus:    [61 83 50 95 93 81 64 67]
-----
Students > 60: 7
Median Score:  74.0
Std Deviation: 15.11
=====
```

## Program 4: Smartwatch Activity

```
import numpy as np

np.random.seed(42)
steps_data = np.random.randint(2000, 15001, 30)

steps_matrix = steps_data.reshape(5, 6)

weekly_avg = np.mean(steps_matrix, axis=1)

max_steps = np.max(steps_matrix)
flat_index = np.argmax(steps_matrix)
week_idx, day_idx = np.unravel_index(flat_index, steps_matrix.shape)

high_activity_days = np.sum(steps_data > 10000)

print("\n=== Smartwatch Activity Report ===")
print("Week\t\tAvg Steps")
print("-----")
for i in range(len(weekly_avg)):
    print(f"Week {i + 1}:\t\t{weekly_avg[i]:.0f}")
print("=====")
print(f"Max Record:\t{max_steps} steps")
print(f"Occurred On:\tWeek {week_idx + 1}, Day {day_idx + 1}")
print("-----")
print(f"Days > 10k:\t{high_activity_days}")
print("=====")
```

### Output:

|                                    |               |
|------------------------------------|---------------|
| === Smartwatch Activity Report === |               |
| Week                               | Avg Steps     |
| -----                              |               |
| Week 1:                            | 8993          |
| Week 2:                            | 7684          |
| Week 3:                            | 9306          |
| Week 4:                            | 6558          |
| Week 5:                            | 9734          |
| =====                              |               |
| Max Record:                        | 14309 steps   |
| Occurred On:                       | Week 5, Day 3 |
| -----                              |               |
| Days > 10k:                        | 10            |
| =====                              |               |

## Program 5: Heart Rate Monitor

```
import numpy as np

np.random.seed(0)
heart_rates = np.random.randint(60, 150, (10, 4))

daily_averages = np.mean(heart_rates, axis=1)

abnormal_counts = np.sum(heart_rates > 120, axis=1)
total_alerts = np.sum(heart_rates > 120)

overall_max = np.max(heart_rates)
overall_min = np.min(heart_rates)

print("\n=== Heart Rate Monitor Log ===")
print("Patient\t\tAvg BPM\t\tAbnormal (>120)")
print("-----")
for i in range(len(daily_averages)):
    print(f"Patient {i + 1}:\t{daily_averages[i]:.1f}\t{abnormal_counts[i]}")
print("=====")
print(f"Overall Max:\t{overall_max} BPM")
print(f"Overall Min:\t{overall_min} BPM")
print(f"Total Alerts:\t{total_alerts}")
print("=====")
```

### Output:

| === Heart Rate Monitor Log === |         |                 |
|--------------------------------|---------|-----------------|
| Patient                        | Avg BPM | Abnormal (>120) |
| -----                          |         |                 |
| Patient 1:                     | 115.5   | 2               |
| Patient 2:                     | 105.0   | 2               |
| Patient 3:                     | 130.2   | 3               |
| Patient 4:                     | 115.8   | 2               |
| Patient 5:                     | 125.0   | 2               |
| Patient 6:                     | 115.0   | 2               |
| Patient 7:                     | 105.2   | 2               |
| Patient 8:                     | 124.8   | 3               |
| Patient 9:                     | 122.0   | 2               |
| Patient 10:                    | 82.8    | 0               |
| =====                          |         |                 |
| Overall Max:                   | 148 BPM |                 |
| Overall Min:                   | 69 BPM  |                 |
| Total Alerts:                  | 20      |                 |
| =====                          |         |                 |

## Program 6: Monthly Sales Analysis

```
import numpy as np

branch_a = [122, 135, 150, 160, 155, 140]
branch_b = [110, 130, 148, 165, 158, 145]

a_array = np.array(branch_a)
b_array = np.array(branch_b)

difference = a_array - b_array

avg_a = np.mean(a_array)
avg_b = np.mean(b_array)

total_a = np.sum(a_array)
total_b = np.sum(b_array)
winner = "Branch A" if total_a > total_b else "Branch B"

correlation = np.corrcoef(a_array, b_array)[0, 1]

print("\n=== Monthly Sales Analysis ===")
print("Month\tBr. A\tBr. B\tDiff (A-B)")
print("-----")
for i in range(len(branch_a)):
    print(f"M-{i + 1}\t{branch_a[i]}\t{branch_b[i]}\t{difference[i]}")
print("=====")
print(f"Avg Sales A:\t{avg_a:.2f}k")
print(f"Avg Sales B:\t{avg_b:.2f}k")
print("-----")
print(f"Best Branch:\t{winner}")
print(f"Correlation:\t{correlation:.4f}")
print("=====")
```

### Output:

| === Monthly Sales Analysis === |       |          |            |
|--------------------------------|-------|----------|------------|
| Month                          | Br. A | Br. B    | Diff (A-B) |
| -----                          |       |          |            |
| M-1                            | 122   | 110      | 12         |
| M-2                            | 135   | 130      | 5          |
| M-3                            | 150   | 148      | 2          |
| M-4                            | 160   | 165      | -5         |
| M-5                            | 155   | 158      | -3         |
| M-6                            | 140   | 145      | -5         |
| =====                          |       |          |            |
| Avg Sales A:                   |       | 143.67k  |            |
| Avg Sales B:                   |       | 142.67k  |            |
| -----                          |       |          |            |
| Best Branch:                   |       | Branch A |            |
| Correlation:                   |       | 0.9812   |            |
| =====                          |       |          |            |

## Program 7: Image Processing

```
import numpy as np

image_data = [
    [100, 120, 130, 90],
    [80, 110, 140, 100],
    [70, 90, 120, 130],
    [110, 140, 150, 160],
]

image_array = np.array(image_data)

normalized_image = image_array / 255.0

transformed_image = image_array * 1.2
final_image = np.clip(transformed_image, 0, 255)

avg_before = np.mean(image_array)
avg_after = np.mean(final_image)

print("\n=== Image Processing Log ===")
print("Original Array (4x4):")
print(image_array)
print("-----")
print("Normalized (First Row Sample):")
print(np.round(normalized_image[0], 2))
print("-----")
print("Transformed & Clipped:")
print(final_image.astype(int))
print("=====")
print(f"Avg Brightness (Pre):\t{avg_before:.2f}")
print(f"Avg Brightness (Post):\t{avg_after:.2f}")
print("=====")
```

### Output:

```
=== Image Processing Log ===
Original Array (4x4):
[[100 120 130  90]
 [ 80 110 140 100]
 [ 70  90 120 130]
 [110 140 150 160]]
-----
Normalized (First Row Sample):
[0.39 0.47 0.51 0.35]
-----
Transformed & Clipped:
[[120 144 156 108]
 [ 96 132 168 120]
 [ 84 108 144 156]
 [132 168 180 192]]
=====
Avg Brightness (Pre):   115.00
Avg Brightness (Post): 138.00
=====
```

## Program 8: Feature Scaling

```
import numpy as np

np.random.seed(42)
dataset = np.random.randint(1, 101, (100, 4))

mean_vals = np.mean(dataset, axis=0)
std_vals = np.std(dataset, axis=0)

scaled_dataset = (dataset - mean_vals) / std_vals

check_means = np.mean(scaled_dataset, axis=0)
check_vars = np.var(scaled_dataset, axis=0)

print("\n=== Feature Scaling Report ===")
print(f"Data Shape:\t{dataset.shape}")
print("-----")
print("Original Data Sample (First 3 rows):")
print(dataset[:3])
print("-----")
print("Verification (Scaled Data):")
print("Means (Target ~0):")
print(np.round(check_means, 10))
print("\nVariances (Target ~1):")
print(np.round(check_vars, 10))
print("=====")
```

### Output:

```
=== Feature Scaling Report ===
Data Shape:      (100, 4)
-----
Original Data Sample (First 3 rows):
[[ 52  93  15  72]
 [ 61  21  83  87]
 [ 75  75  88 100]]
-----
Verification (Scaled Data):
Means (Target ~0):
[-0.  0.  0. -0.]

Variances (Target ~1):
[1.  1.  1.  1.]
=====
```

## Program 9: Traffic Control

```
import numpy as np

np.random.seed(42)

traffic_data = np.random.randint(0, 80, (24, 6))

hourly_averages = np.mean(traffic_data, axis=1)

total_density_per_sensor = np.sum(traffic_data, axis=0)
busiest_sensor_idx = np.argmax(total_density_per_sensor)

capped_data = np.minimum(traffic_data, 50)

energy = np.sum(capped_data * 0.5)

print("\n=== Traffic Control Log ===")
print(f"Sensors Active:\t6")
print(f"Duration:\t24 Hours")
print("-----")
print("Hourly Averages (First 5h):")
print(np.round(hourly_averages[:5], 1))
print("-----")
print(f"Busiest Sensor:\tSensor {busiest_sensor_idx + 1}")
print(f"Total Traffic:\t{total_density_per_sensor[busiest_sensor_idx]}")
print("-----")
print(f"Total Energy:\t{energy:.1f} Units")
print("=====")
```

### Output:

```
=== Traffic Control Log ===
Sensors Active:6
Duration:      24 Hours
-----
Hourly Averages (First 5h):
[48.3 28.8 34.8 48.5 52.5]
-----
Busiest Sensor:Sensor 1
Total Traffic: 1030
-----
Total Energy:  2379.5 Units
=====
```



## Program 10: Mechanical System

```
import numpy as np

F = np.array([[5, 2, 1], [3, 1, 4], [6, 3, 2], [4, 2, 5], [7, 1, 3]])

T = np.array([[2, 1, 0], [1, 3, 1], [0, 2, 2]])

F_new = np.dot(F, T)

magnitudes = np.linalg.norm(F_new, axis=1)

max_idx = np.argmax(magnitudes)
max_val = magnitudes[max_idx]

eigenvalues, eigenvectors = np.linalg.eig(T)

# Output Receipt
print("\n=== Mechanical System Analysis ===")
print(f"Components:\t{F.shape[0]}")
print("-----")
print("Transformed Forces (F_new):")
print(F_new)
print("-----")
print("Magnitudes:")
print(np.round(magnitudes, 2))
print("-----")
print(f"Highest Stress:\tComponent {max_idx + 1}")
print(f"Max Force:\t{max_val:.2f} N")
print("=====")
print("Transformation Eigenvalues:")
print(np.round(eigenvalues, 4))
print("=====")
```

### Output:

```
=== Mechanical System Analysis ===
Components:      5
-----
Transformed Forces (F_new):
[[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
-----
Magnitudes:
[18.14 18.06 25.2  25.38 23.02]
-----
Highest Stress:Component 4
Max Force:      25.38 N
=====
Transformation Eigenvalues:
[4.3028 2.      0.6972]
=====
```

---

## Lab 4

### “Assignment”

#### Program 1: Basic Python Operations

```
def calculations(n1, n2):
    summ = n1 + n2
    diff = n1 - n2
    prod = n1 * n2
    if n2 != 0:
        div = n1 / n2
    else:
        div = 0
    return summ, diff, prod, div

def to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

def display(n1, n2, calculations, greater, c,
f): summ, diff, prod, div = calculations

    print("\n=== Results ===")
    print("-----")
    print(f"Sum:\t\t\t", summ)
    print(f"Difference:\t\t", diff)
    print(f"Product:\t\t", prod)
    print(f"Division:\t\t {div:.2f}")
    print("-----")
    print(f"Greater Number:\t\t", greater)
    print("\n=== Temperature ===")
    print(f"Celsius:\t\t {c}°C")
    print(f"Fahrenheit:\t\t {f}°F")
    print("=====")
```

```

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
temp_c = float(input("Enter Celsius Temp: "))

calcs = calculations(num1, num2)
greater_val = max(num1, num2)
temp_f = to_fahrenheit(temp_c)

display(num1, num2, calcs, greater_val, temp_c, temp_f)

```

### Output:

```

Enter first number: 19
Enter second number: 17
Enter Celsius Temp: 47

=== Results ===
-----
Sum:                               36.0
Difference:                        2.0
Product:                           323.0
Division:                          1.12
-----
Greater Number:                    19.0

=== Temperature ===
Celsius:                          47.0°C
Fahrenheit:                        116.6°F
=====

```

## Program 2: Control-Flow Mini-Project

```
def category(age):
    if age < 13:
        return "Child"
    elif age >= 13 and age <= 19:
        return "Teenager"
    else:
        return "Adult"

def display(age, category):
    print("\n=== Age Report ===")
    print(f"Age Entered:\t\t", age)
    print("-----")
    print(f"Category:\t\t", category)
    print("=====")

while True:
    user_input = input("Enter your age (or 'exit'): ")

    if user_input == "exit":
        print("Exiting program...")
        break

    age = int(user_input)

    if age < 0:
        print("Age cannot be negative!")
        continue

    category = category(age)
    display(age, category)
```

## Output:

```
Enter your age (or 'exit'): 19

=== Age Report ===
Age Entered:           19
-----
Category:              Teenager
=====
Enter your age (or 'exit'): 17

=== Age Report ===
Age Entered:           17
-----
Category:              Teenager
=====
Enter your age (or 'exit'): exit
Exiting program...
```

### Program 3: Class for AI Agent

```
class AIAgent:
    def __init__(self, name, model, version):
        self.name = name
        self.model = model
        self.version = version

    def introduce(self):
        print("\n=== Agent Details ===")
        print(f"Agent Name:\t\t", self.name)
        print(f"Model:\t\t\t\t", self.model)
        print(f"Version:\t\t", self.version)
        print("=====")

    def predict(self, input_value):
        return input_value * 2.5

def display(val, result):
    print("\n=== Prediction Result ===")
    print(f"Input Value:\t\t", val)
    print("-----")
    print(f"Model Output:\t\t", result)
    print("=====")

my_agent = AIAgent("Hitler", "Killer", 1.0)

my_agent.introduce()

user_val = float(input("\nEnter value for prediction: "))
prediction = my_agent.predict(user_val)

display(user_val, prediction)
```

## Output:

```
=== Agent Details ===
Agent Name:           Hitler
Model:                Killer
Version:              1.0
=====

Enter value for prediction: 19

=== Prediction Result ===
Input Value:          19.0
-----
Model Output:         47.5
=====
```

## Program 4: Inheritance Problem

```
class Dataset:
    def __init__(self, data_list):
        self.data = data_list

    def summary(self):
        print("\n=== Dataset Summary ===")
        print(f"Type:\t\t Base Dataset")
        print(f"Total Elements:\t", len(self.data))
        print("=====")

class ImageDataset(Dataset):
    def __init__(self, data_list, image_size):
        super().__init__(data_list)
        self.image_size = image_size

    def summary(self):
        print("\n=== Image Dataset Info ===")
        print(f"Type:\t\t Child Class")
        print(f"Total Images:\t", len(self.data))
        print(f"Resolution:\t", self.image_size)
        print("=====")

def show_info(dataset_obj):
    dataset_obj.summary()

text_data = ["Text1", "Text2", "Text3", "Text4"]
base_set = Dataset(text_data)

img_data = ["img1.jpg", "img2.jpg"]
img_set = ImageDataset(img_data, "1920x1080")

print("\n--- Processing Base Class ---")
show_info(base_set)

print("\n--- Processing Child Class ---")
show_info(img_set)
```



## Output:

```
--- Processing Base Class ---  
  
=== Dataset Summary ===  
Type:          Base Dataset  
Total Elements: 4  
=====
```

```
--- Processing Child Class ---  
  
=== Image Dataset Info ===  
Type:          Child Class  
Total Images:  2  
Resolution:    1920x1080  
=====
```

## Program 5: Linked List

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            print(f"Inserted:\t {value} (New Head)")
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
        print(f"Inserted:\t {value}")

    def delete_value(self, value):
        print(f"\nAttempting to delete: {value}")

        if self.head is None:
            print("List is empty!")
            return

        if self.head.data == value:
            self.head = self.head.next
            print("Result:\t\t Deleted Head Node")
            return
```

```

        temp = self.head
        while temp.next:
            if temp.next.data == value:
                temp.next = temp.next.next
                print("Result:\t\t Value Deleted")
                return
            temp = temp.next

        print("Result:\t\t Value Not Found")

    def display(self):
        print("\n=== Current List Status ===")
        if self.head is None:
            print("List is Empty")
        else:
            temp = self.head
            print("HEAD", end=" -> ")
            while temp:
                print(f"[{temp.data}]", end=" -> ")
                temp = temp.next
            print("NULL")
        print("=====")

ll = LinkedList()

print("\n--- Insertion Phase ---")
ll.insert_at_end(10)
ll.insert_at_end(20)
ll.insert_at_end(30)
ll.insert_at_end(40)
ll.insert_at_end(50)

ll.display()

ll.delete_value(30)
ll.display()

ll.delete_value(10)
ll.display()
ll.delete_value(99)

```

## Output:

```
--- Insertion Phase ---
Inserted:    10 (New Head)
Inserted:    20
Inserted:    30
Inserted:    40
Inserted:    50

=== Current List Status ===
HEAD -> [10] -> [20] -> [30] -> [40] -> [50] -> NULL
=====

Attempting to delete: 30
Result:      Value Deleted

=== Current List Status ===
HEAD -> [10] -> [20] -> [40] -> [50] -> NULL
=====

Attempting to delete: 10
Result:      Deleted Head Node

=== Current List Status ===
HEAD -> [20] -> [40] -> [50] -> NULL
=====

Attempting to delete: 99
Result:      Value Not Found
```

## Program 6: Sorting & Searching for AI Data

```
def bubble_sort(arr):
    sorted_arr = arr[:]
    n = len(sorted_arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if sorted_arr[j] > sorted_arr[j + 1]:
                sorted_arr[j], sorted_arr[j + 1] = sorted_arr[j +
1], sorted_arr[j]
                swapped = True

        if not swapped:
            break

    return sorted_arr

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        mid_val = arr[mid]

        if mid_val == target:
            return mid # Found
        elif mid_val < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

```

def print_results(original, sorted_data, target, index):
    print("\n=== Data Processing Report ===")
    print(f"Original Data:\t {original}")
    print(f"Sorted Data:\t {sorted_data}")
    print("-----")
    print(f"Search Target:\t {target}")

    if index != -1:
        print(f"Status:\t\t Found")
        print(f"Index Location:\t {index}")
    else:
        print(f"Status:\t\t Not Found")
    print("=====")

raw_data = [45, 12, 78, 34, 2, 89, 33]

sorted_list = bubble_sort(raw_data)

while True:
    search_target = int(input("\nEnter number to search: "))
    break

result_index = binary_search(sorted_list, search_target)

print_results(raw_data, sorted_list, search_target, result_index)

```

## Output:

```

Enter number to search: 19

=== Data Processing Report ===
Original Data:  [45, 12, 78, 34, 2, 89, 33]
Sorted Data:    [2, 12, 33, 34, 45, 78, 89]
-----
Search Target:  19
Status:         Not Found
=====

```

## Program 7: Mini AI Dataset Processor

```
def process_scores(score_list):
    unique_scores = list(set(score_list))

    unique_scores.sort(reverse=True)

    top_three = unique_scores[:3]

    return unique_scores, top_three

def print_score_report(original, unique, top_3):
    print("\n=== Confidence Score Report ===")
    print(f"Total Inputs:\t\t", len(original))
    print(f"Unique Scores:\t\t", len(unique))
    print("-----")
    print("Top 3 Scores:")

    rank = 1
    for score in top_3:
        print(f"Rank {rank}:\t\t {score}")
        rank += 1

    print("=====")

raw_data = [0.88, 0.45, 0.95, 0.88, 0.12, 0.99, 0.95, 0.76,
0.99]
print(f"Raw Data: {raw_data}")

cleaned_data, best_scores = process_scores(raw_data)

print_score_report(raw_data, cleaned_data, best_scores)
```

### Output:

```
Raw Data: [0.88, 0.45, 0.95, 0.88, 0.12, 0.99, 0.95, 0.76, 0.99]

=== Confidence Score Report ===
Total Inputs:          9
Unique Scores:         6
-----
Top 3 Scores:
Rank 1:                0.99
Rank 2:                0.95
Rank 3:                0.88
=====
```

---

## Lab No 5

### “Seaborn, Matplotlib, Sklearn”

#### Program 1: Patient Stay Analysis (Healthcare)

##### Importing Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

✓ 20.4s

Python

##### Sample Data

```
data = {
    'disease_type': ['Flu', 'Flu', 'Covid-19', 'Covid-19', 'Fracture', 'Fracture'],
    'hospital_stay_days': [3, 4, 12, 14, 5, 7],
    'age': [25, 30, 45, 50, 20, 22]
}
```

✓ 0.0s

Python

##### Convert to a DataFrame

```
df = pd.DataFrame(data)
```

✓ 0.0s

Python

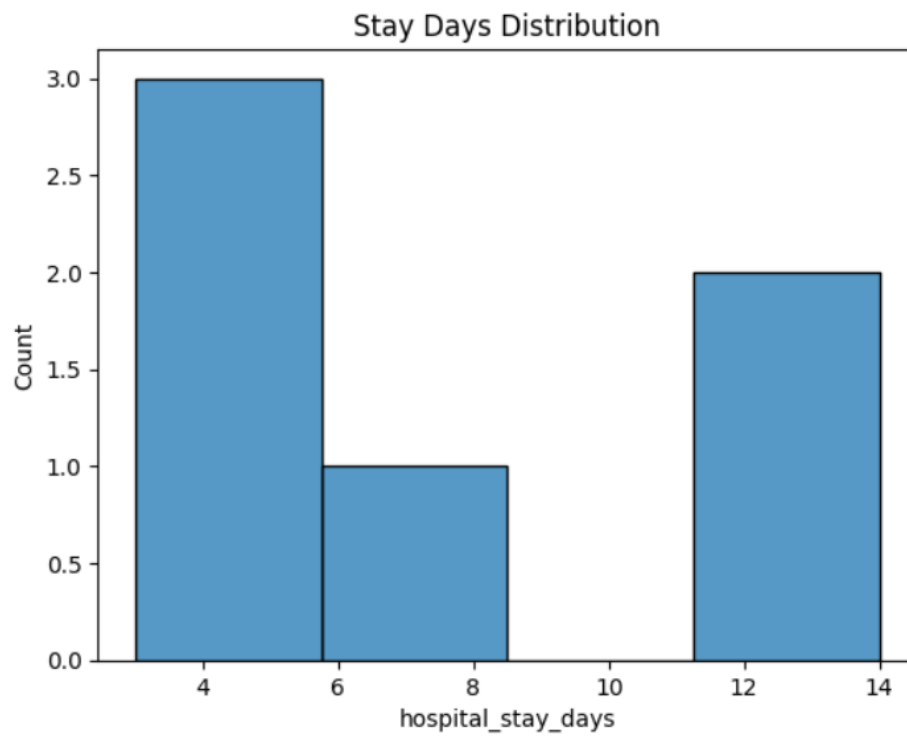


## Plot distribution of stay days

```
sns.histplot(df['hospital_stay_days'])  
plt.title('Stay Days Distribution')  
plt.show()
```

✓ 0.7s

Python

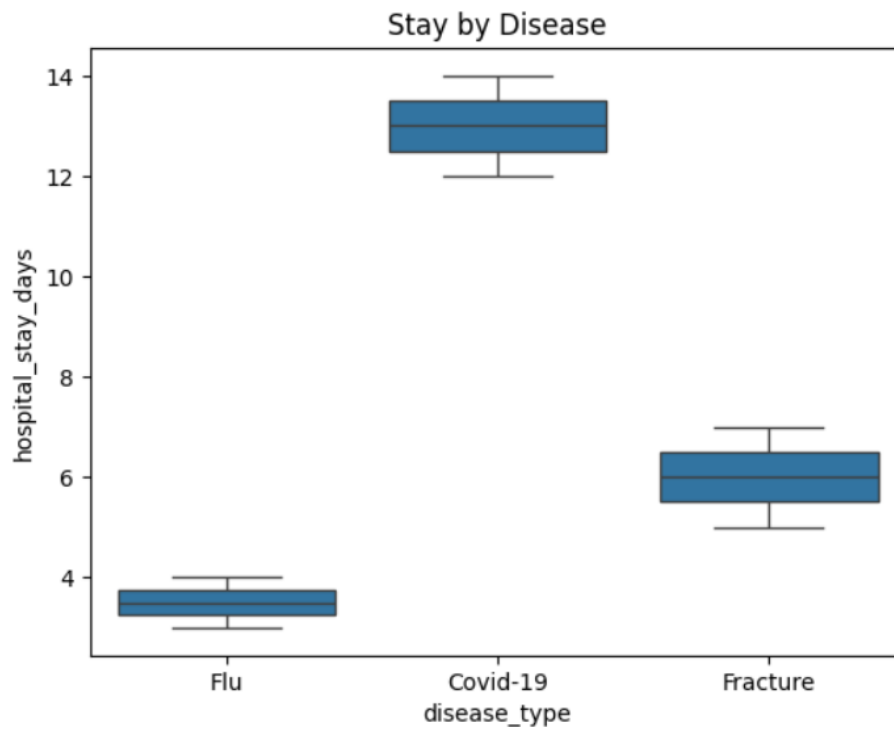


## Compare stay across diseases

```
sns.boxplot(x='disease_type', y='hospital_stay_days', data=df)
plt.title('Stay by Disease')
plt.show()
```

✓ 0.1s

Python



## Find the highest median stay

```
medians = df.groupby('disease_type')['hospital_stay_days'].median()
print("Median Stays:")
print(medians)
```

✓ 0.0s

Python

```
Median Stays:
disease_type
Covid-19    13.0
Flu         3.5
Fracture     6.0
Name: hospital_stay_days, dtype: float64
```

```
# Prints the disease with the highest number
print(f"\nHighest Median Stay: {medians.idxmax()}")
```

✓ 0.0s

Python

Highest Median Stay: Covid-19

## Program 2: Sales Distribution Comparison (Business)

### Importing Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

✓ 5.4s

Python

### Create simple data

```
data = {
    'region': ['North']*10 + ['South']*10 + ['East']*10 + ['West']*10,
    'monthly_sales': np.concatenate([
        np.random.normal(5000, 200, 10), # North: Low spread
        np.random.normal(6000, 1500, 10), # South: High spread
        np.random.normal(4000, 300, 10), # East: Low spread
        np.random.normal(5500, 800, 10) # West: Medium spread
    ])
}
```

```
df = pd.DataFrame(data)
```

✓ 0.0s

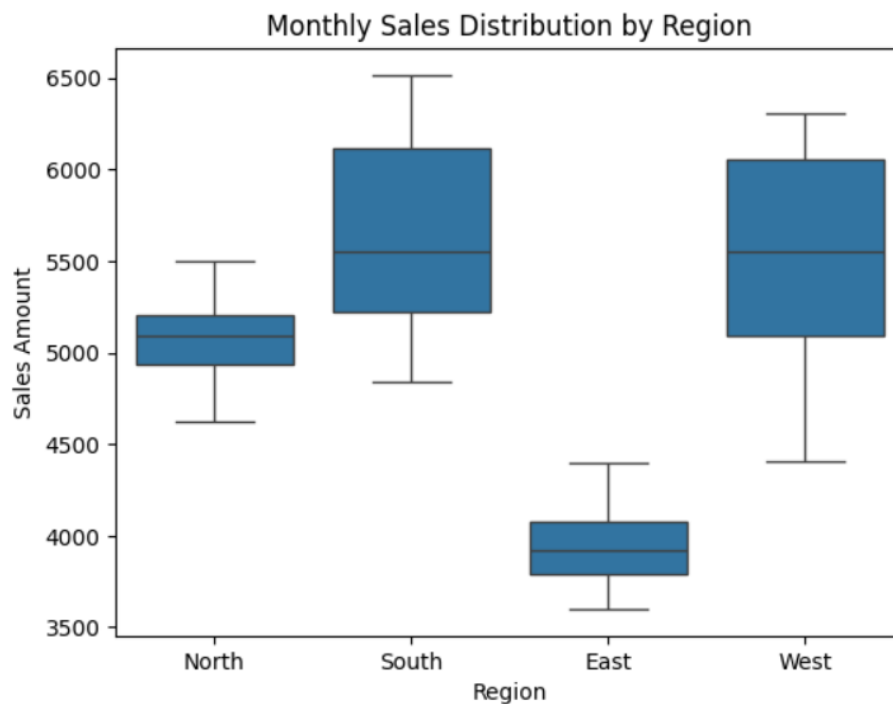
Python

## Visualize sales distribution per region

```
sns.boxplot(x='region', y='monthly_sales', data=df)
plt.title('Monthly Sales Distribution by Region')
plt.xlabel('Region')
plt.ylabel('Sales Amount')
plt.show()
```

✓ 0.1s

Python



## Detect regions with high sales variability

```
variability = df.groupby('region')['monthly_sales'].std()

print("Sales Variability (Standard Deviation):")
print(variability)
print(f"\nRegion with highest variability: {variability.idxmax()}")
```

✓ 0.0s

Python

```
Sales Variability (Standard Deviation):
region
East      249.838986
North     271.165106
South     557.931504
West      685.462949
Name: monthly_sales, dtype: float64
```

```
Region with highest variability: West
```

## Program 3: Student Performance Correlation (Education)

### Importing Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

✓ 1.2s

Python

### Create simple data

```
data = {
    'study_hours': [2, 3, 5, 7, 8, 9, 10, 1],
    'attendance': [70, 80, 85, 90, 95, 98, 99, 60],
    'final_score': [60, 65, 75, 85, 90, 95, 98, 55]
}

df = pd.DataFrame(data)
```

✓ 0.0s

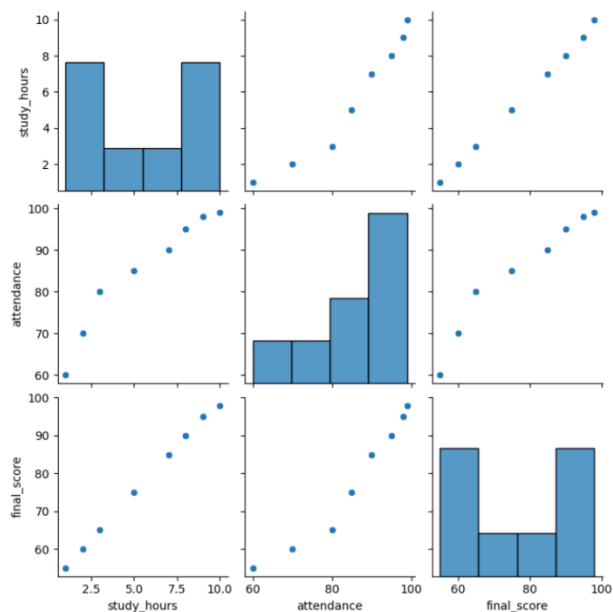
Python

### Create a pair plot

```
sns.pairplot(df)
plt.savefig('pair_plot.png')
plt.show()
```

✓ 1.1s

Python



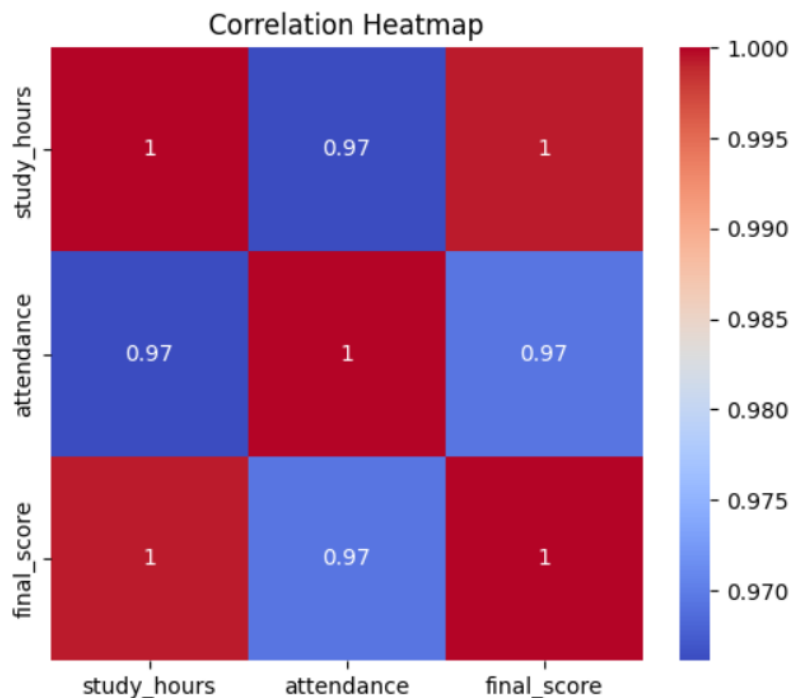
## Plot a correlation heatmap

```
plt.figure(figsize=(6, 5))

corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.savefig('heatmap.png')
plt.show()
```

✓ 0.2s

Python



## Interpret which feature most affects the final score

```
correlations = corr_matrix['final_score'].drop('final_score')

print("Correlations with Final Score:")
print(correlations)
print(f"\nFeature that most affects Final Score: {correlations.idxmax()}")
```

✓ 0.0s

Python

Correlations with Final Score:

study\_hours 0.999335

attendance 0.969298

Name: final\_score, dtype: float64

Feature that most affects Final Score: study\_hours

## Program 4: Customer Segmentation Visualization

### Importing Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

✓ 1.2s

Python

### Create simple data

```
data = {
    'annual_income': [
        25, 28, 30, 22, 26, # Low Income
        25, 28, 30, 24, 27, # Low Income
        80, 85, 90, 88, 82, # High Income
        80, 85, 90, 88, 82, # High Income
        50, 55, 60, 52, 58 # Medium Income
    ],
    'spending_score': [
        10, 15, 20, 12, 18, # Low Spend
        80, 85, 90, 82, 88, # High Spend
        10, 15, 20, 12, 18, # Low Spend
        80, 85, 90, 82, 88, # High Spend
        50, 55, 60, 52, 58 # Medium Spend
    ],
    'age': [
        45, 50, 48, 52, 49,
        20, 22, 19, 21, 23,
        40, 45, 42, 48, 44,
        25, 28, 26, 29, 27,
        35, 38, 32, 36, 34]
}
```

```
df = pd.DataFrame(data)
```

✓ 0.0s

Python

## Visualize relationships using Scatter Plot

```
plt.figure(figsize=(8, 6))

# Plot Income vs Spending, colored by Spending Score
sns.scatterplot(
    x='annual_income',
    y='spending_score',
    hue='spending_score', # Color-code points
    palette='coolwarm',   # Blue (Low) to Red (High)
    data=df,
    s=100                 # Size of dots
)

plt.title('Customer Segments: Income vs Spending')
plt.xlabel('Annual Income ($k)')
plt.ylabel('Spending Score (1-100)')
plt.show()

print("Potential Segments Identified:")
print("1. Low Income, High Spending (Top Left)")
print("2. High Income, High Spending (Top Right)")
print("3. Medium Income, Medium Spending (Center)")
print("4. Low Income, Low Spending (Bottom Left)")
print("5. High Income, Low Spending (Bottom Right)")
```

✓ 0.1s

Python



Potential Segments Identified:

1. Low Income, High Spending (Top Left)
2. High Income, High Spending (Top Right)
3. Medium Income, Medium Spending (Center)
4. Low Income, Low Spending (Bottom Left)
5. High Income, Low Spending (Bottom Right)



## Program 5: Model Result Visualization

### Importing Libraries

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
```

✓ 1.6s

Python

### Create simple result data

```
y_true = ['Cat', 'Dog', 'Cat', 'Cat', 'Dog', 'Dog', 'Cat', 'Dog', 'Cat', 'Dog']
# What the model predicted
y_pred = ['Cat', 'Cat', 'Cat', 'Cat', 'Dog', 'Dog', 'Dog', 'Dog', 'Cat', 'Dog']
```

✓ 0.0s

Python

### Calculate the Confusion Matrix

```
labels = ['Cat', 'Dog']
cm = confusion_matrix(y_true, y_pred, labels=labels)
```

✓ 0.0s

Python

### Plot the Heatmap

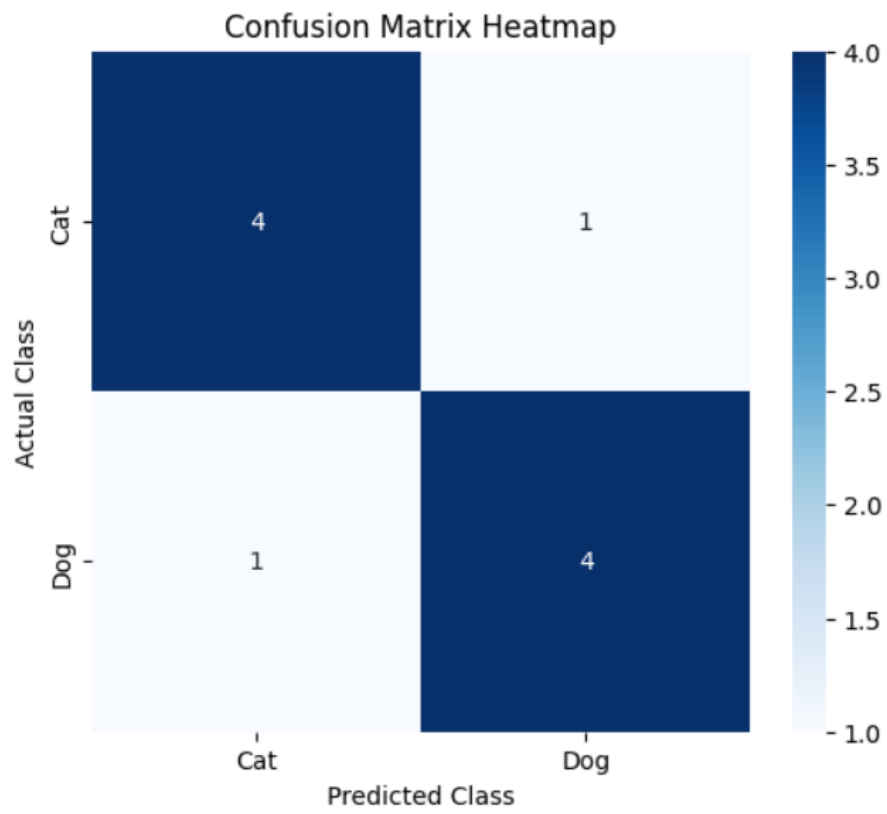
```
plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=labels, yticklabels=labels)

plt.ylabel('Actual Class')
plt.xlabel('Predicted Class')
plt.title('Confusion Matrix Heatmap')
plt.show()
```

✓ 0.2s

Python



## Program 6: Telecom Customer Churn Prediction

### Importing Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

✓ 1.1s

Python

### Create simple data

```
data = {
    'tenure': [1, 2, 5, 50, 60, 55, 3, 45, 2, 52],
    'monthly_charges': [70, 80, 85, 40, 50, 45, 90, 42, 75, 48],
    'contract_type': ['Month', 'Month', 'Month', 'Year', 'Year', 'Year', 'Month',
                     'Year', 'Month', 'Year'],
    'churn': [1, 1, 1, 0, 0, 0, 1, 0, 1, 0] # 1 = Left (Churn), 0 = Stayed
}

df = pd.DataFrame(data)
```

✓ 0.0s

Python

### Encode categorical variables

```
df_ready = pd.get_dummies(df, columns=['contract_type'], drop_first=True)

# Separate features (X) and target (y)
X = df_ready.drop('churn', axis=1)
y = df_ready['churn']
```

✓ 0.0s

Python

### Split data for Training

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ 0.0s

Python

### Train Logistic Regression model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

✓ 0.0s

Python

▼ LogisticRegression ⓘ ?  
► Parameters

## Evaluate the model

```
predictions = model.predict(X_test)

print("Model Accuracy:", accuracy_score(y_test, predictions))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

# Show specific comparison
print("\nPrediction vs Actual:")
results = pd.DataFrame({'Predicted': predictions, 'Actual': y_test.values})
print(results)
```

✓ 0.0s

Python

Model Accuracy: 1.0

Confusion Matrix:

[[2]]

Prediction vs Actual:

|   | Predicted | Actual |
|---|-----------|--------|
| 0 | 1         | 1      |
| 1 | 1         | 1      |

## Program 7: House Price Prediction

### Importing Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

✓ 1.0s

Python

### Create simple data

```
data = {
    'area': [1000, 1500, 2000, 1200, 1800, 2500, 800, 3000, 1400, 2200],
    'bedrooms': [2, 3, 4, 2, 3, 4, 1, 5, 2, 4],
    'location_score': [5, 7, 8, 6, 8, 9, 4, 10, 5, 7],
    'price': [150000, 220000, 300000, 170000, 250000, 350000, 120000, 400000, 180000, 320000]
}

df = pd.DataFrame(data)
```

✓ 0.0s

Python

### Split data for Training

```
x = df[['area', 'bedrooms', 'location_score']]
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ 0.0s

Python

### Train Logistic Regression model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

✓ 0.0s

Python

▼ LinearRegression ⓘ ?  
► Parameters

## Predict prices and calculate Mean Squared Error

```
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)

print(f"Mean Squared Error: {mse:.2f}")

print("\nPredictions vs Actual:")
results = pd.DataFrame({'Predicted': predictions, 'Actual': y_test.values})
print(results)
```

✓ 0.0s

Python

Mean Squared Error: 155575369.27

Predictions vs Actual:

|   | Predicted     | Actual |
|---|---------------|--------|
| 0 | 197303.370787 | 180000 |
| 1 | 223426.966292 | 220000 |

## Program 8: Disease Classification System

### Importing Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
```

✓ 1.1s

Python

### Create simple data

```
data = {
    'glucose': [85, 160, 100, 180, 90, 175, 110, 190, 88, 165],
    'BMI': [22, 35, 24, 38, 23, 36, 25, 40, 21, 34],
    'age': [25, 50, 30, 55, 28, 60, 32, 65, 22, 52],
    'outcome': [0, 1, 0, 1, 0, 1, 0, 1, 0, 1] # 0 = Healthy, 1 = Disease
}

df = pd.DataFrame(data)
```

✓ 0.0s

Python

### Split Features and Target

```
x = df[['glucose', 'BMI', 'age']]
y = df['outcome']
```

✓ 0.0s

Python

### Normalize the data

```
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
```

✓ 0.0s

Python

### Split data for Training

```
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2,
                                                    random_state=42)
```

✓ 0.0s

Python

## Train Decision Tree Classifier

```
model = DecisionTreeClassifier()  
model.fit(X_train, y_train)
```

✓ 0.0s

Python

▼ DecisionTreeClassifier ⓘ ?  
► Parameters

## Evaluate Precision, Recall, and F1-score

```
predictions = model.predict(X_test)  
  
print("Classification Report:")  
# target_names adds labels to the report for clarity  
print(classification_report(y_test, predictions, target_names=['Healthy', 'Disease']))  
  
# Show specific prediction comparison  
print("Predictions vs Actual:")  
results = pd.DataFrame({'Predicted': predictions, 'Actual': y_test.values})  
print(results)
```

✓ 0.0s

Python

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Healthy      | 1.00      | 1.00   | 1.00     | 1       |
| Disease      | 1.00      | 1.00   | 1.00     | 1       |
| accuracy     |           |        | 1.00     | 2       |
| macro avg    | 1.00      | 1.00   | 1.00     | 2       |
| weighted avg | 1.00      | 1.00   | 1.00     | 2       |

Predictions vs Actual:

|   | Predicted | Actual |
|---|-----------|--------|
| 0 | 0         | 0      |
| 1 | 1         | 1      |



## Program 9: Email Spam Detection

### Importing Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
```

✓ 1.7s

Python

### Create simple data

```
emails = [
    "Win a free lottery prize now", # Spam
    "Meeting regarding the project", # Ham (Not Spam)
    "Claim your free money offer", # Spam
    "Project report is due tomorrow", # Ham
    "Limited time offer apply now", # Spam
    "Lets grab lunch later" # Ham
]
labels = ['spam', 'ham', 'spam', 'ham', 'spam', 'ham']
```

✓ 0.0s

Python

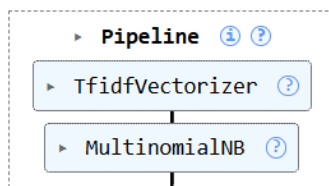
### Build the model pipeline

```
model = make_pipeline(TfidfVectorizer(), MultinomialNB())

# Train the model
model.fit(emails, labels)
```

✓ 0.0s

Python



## Test on new email text

```
new_emails = [  
    "Hey, are we still meeting for dinner?",  
    "Congratulations you won a free prize!"  
]  
  
# Predict  
predictions = model.predict(new_emails)  
  
print("Spam Detection Results:")  
for email, label in zip(new_emails, predictions):  
    print(f"Email: '{email}' -> Prediction: {label}")
```

✓ 0.0s

Python

Spam Detection Results:

Email: 'Hey, are we still meeting for dinner?' -> Prediction: ham

Email: 'Congratulations you won a free prize!' -> Prediction: spam

## Program 10: Student Risk Prediction

### Importing Libraries

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
```

✓ 1.0s

Python

### Create simple data

```
data = {
    'attendance': [90, 85, 40, 50, 95, 80, 45, 60, 30, 88],
    'mid_marks': [80, 75, 30, 40, 85, 70, 35, 55, 20, 78],
    'final_marks': [85, 78, 35, 45, 90, 75, 38, 58, 25, 82],
    'risk_status': [0, 0, 1, 1, 0, 0, 1, 1, 1, 0] # 0 = Safe, 1 = At Risk
}

df = pd.DataFrame(data)
```

✓ 0.0s

Python

### Split Features and Target

```
X = df[['attendance', 'mid_marks', 'final_marks']]
y = df['risk_status']
```

✓ 0.0s

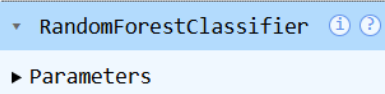
Python

### Build and Train Random Forest Classifier

```
model = RandomForestClassifier(n_estimators=10, random_state=42)
model.fit(X, y)
```

✓ 0.0s

Python



## Predict for new students

```
new_students = pd.DataFrame({
    'attendance': [92, 40],
    'mid_marks': [85, 35],
    'final_marks': [88, 30]
})

predictions = model.predict(new_students)
labels = ['Safe', 'At Risk']

print("Risk Predictions:")
for i, pred in enumerate(predictions):
    print(f"Student {i+1}: {labels[pred]}")
```

✓ 0.0s

Python

Risk Predictions:  
Student 1: Safe  
Student 2: At Risk

## Display Feature Importance

```
importances = model.feature_importances_
print("\nFeature Importance (0 to 1):")
for feature, imp in zip(X.columns, importances):
    print(f"{feature}: {imp:.2f}")
```

✓ 0.0s

Python

Feature Importance (0 to 1):  
attendance: 0.30  
mid\_marks: 0.30  
final\_marks: 0.40